MATH 8090: Spectral Analysis of Time Series II

Whitney Huang, Clemson University

11/2-6/2026

Contents

Spetral ANOVA Example . . . 2
SOI Example . . . 2
Plot the time series and ACF . . . 2
Raw periodogram . . . 3
Averaged periodogram . . . 5
Smoothed periodogram . . . 7
SOI tapering . . . 10
Parametric Spectral Estimation . . . 11
Setup . . . 11
AR(1): Persistence Appears as Low-Frequency Variation . . . 11
ARMA(1,1): Using an AR Model as an Approximation . . . 12
AR(2): Resolving a Narrow Spectral Peak . . . 13
SOI Example: Choosing the AR Order . . . 14
How Strongly Do the Criteria Favor the Selected Order? . . . 16
Lagged regression . . . 17
SOI and Recruitment Time Series . . . 18
Time-Domain Approach . . . 21
Building a Simple Dynamic Regression Model . . . 23
Estimating the Cross-Spectrum and Coherence . . . 28
Lagged Regression in the Frequency Domain . . . 30
Example: SOI and Recruitment . . . 31
Interpreting the Estimated Response . . . 36
Two-Sided Filtering versus Causal Prediction . . . 36
Suggested Extensions . . . 37

Spetral ANOVA Example

Consider five observations. There are two positive Fourier frequencies, $\omega_1 = 1/5$ and $\omega_2 = 2/5$, corresponding to periods of 5 and 2.5 observation intervals.

The following code fits the two sine–cosine pairs and verifies that their regression sums of squares equal twice the corresponding periodogram ordinates.

```
y <- c(1, 2, 3, 2, 1)
n <- length(y)
t <- seq_len(n)

# Each model term contains a cosine and a sine.
wave1 <- cbind(cos(2 * pi * (t / n)), sin(2 * pi * (t / n)))
wave2 <- cbind(cos(2 * pi * (2 * t / n)), sin(2 * pi * (2 * t / n)))
fit <- lm(y ~ wave1 + wave2)

# ANOVA-style decomposition; no F-tests.
ss <- c(sum((fitted(lm(y ~ wave1)) - mean(y))^2),
        sum(residuals(lm(y ~ wave1))^2) - sum(residuals(fit)^2))

# Periodogram at the two positive Fourier frequencies.
d <- fft(y - mean(y)) / sqrt(n)
I <- Mod(d[2:3])^2

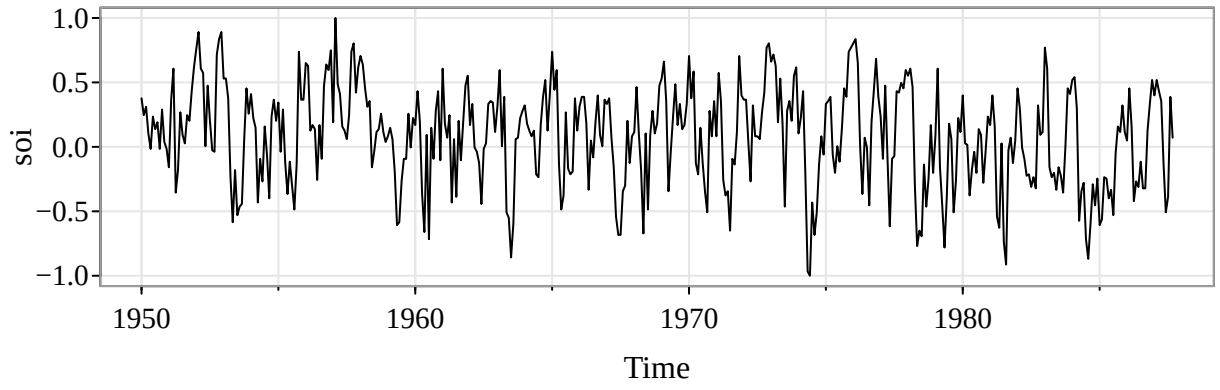
data.frame(frequency = c(1, 2) / n, df = 2,
regression_SS = ss, twice_periodogram = 2 * I, fraction = ss / sum((y - mean(y))^2))
```

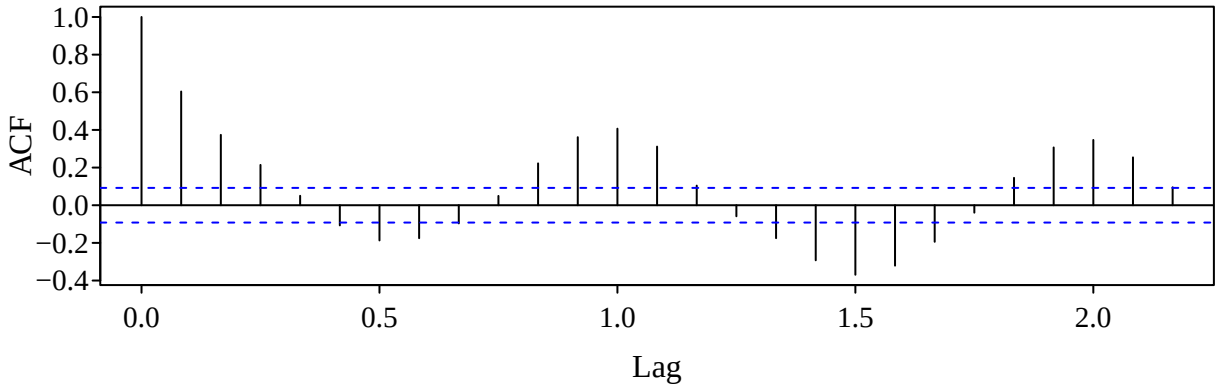
```
##   frequency df regression_SS twice_periodogram   fraction
## 1       0.2  2    2.74164079        2.74164079 0.97915742
## 2       0.4  2    0.05835921        0.05835921 0.02084258
```

SOI Example

Plot the time series and ACF

```
library(astsa)
par(mgp = c(2.2, 0.8, 0), mar = c(3.5, 4, 0.8, 0.6), las = 1,
    mfrow = c(2, 1), family = "serif")
tsplot(soi)
acf(soi, main = "")
```



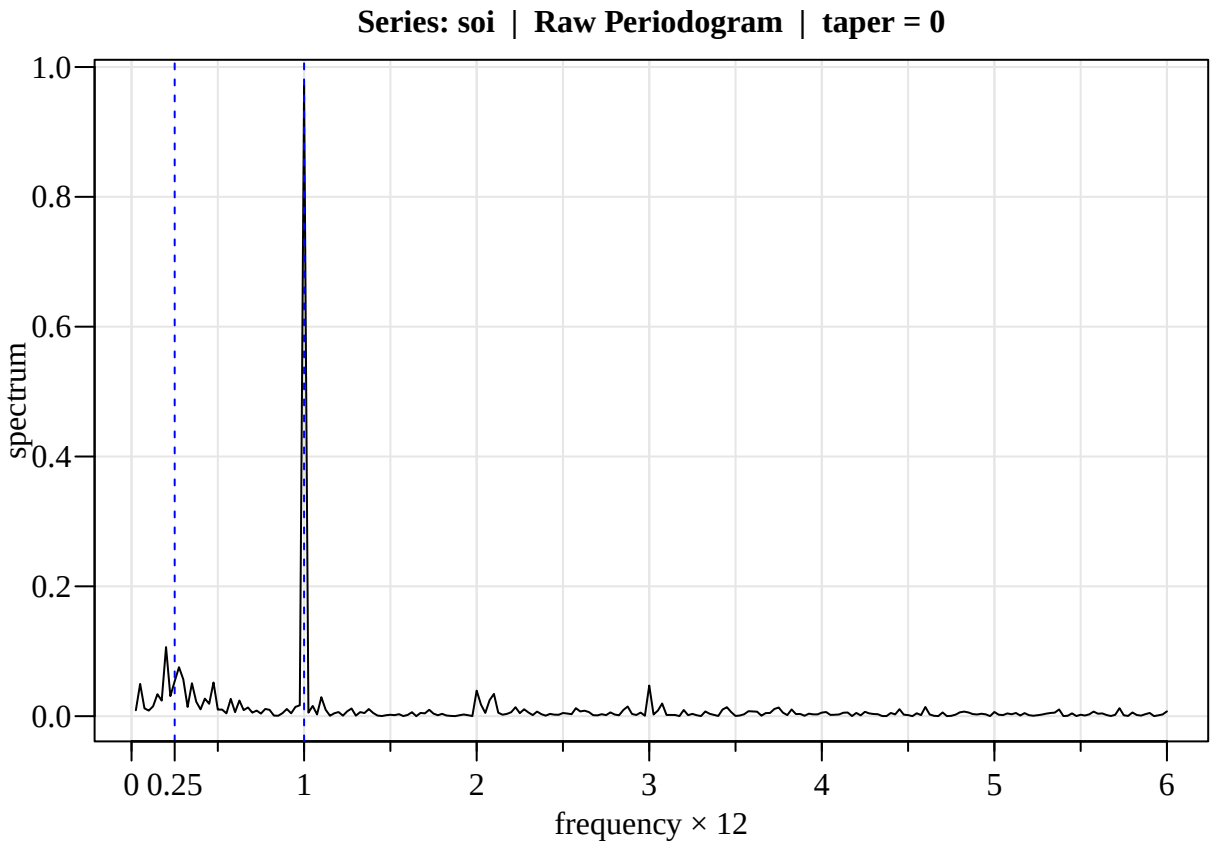


Raw periodogram

An approximate $100(1-\alpha)\%$ confidence interval for $f(\omega)$

$$\frac{2I(\omega_j)}{\chi^2_2(1-\alpha/2)} \leq f(\omega) \leq \frac{2I(\omega_j)}{\chi^2_2(\alpha/2)}$$

```
par(mgp = c(2.2, 1, 0), mar = c(3.5, 4, 1.4, 0.6), las = 1, family = "serif")
soi.per <- mvspec(soi)
abline(v = c(1 / 4, 1), lty = 2, col = "blue")
axis(1, at = 0.25)
```




```
U = qchisq(.025, 2)
L = qchisq(.975, 2)
# 4-year period
soi.per$details[10,]
```

```
## frequency    period  spectrum
##    0.2500    4.0000    0.0537
```

```
c(2 * soi.per$spec[10] / L, 2 * soi.per$spec[10] / U)
```

```
## [1] 0.0145653 2.1222066
```

```
# 1-year period
soi.per$details[40,]
```

```
## frequency    period  spectrum
##    1.0000    1.0000    0.9722
```

```
c(2 * soi.per$spec[40] / L, 2 * soi.per$spec[40] / U)
```

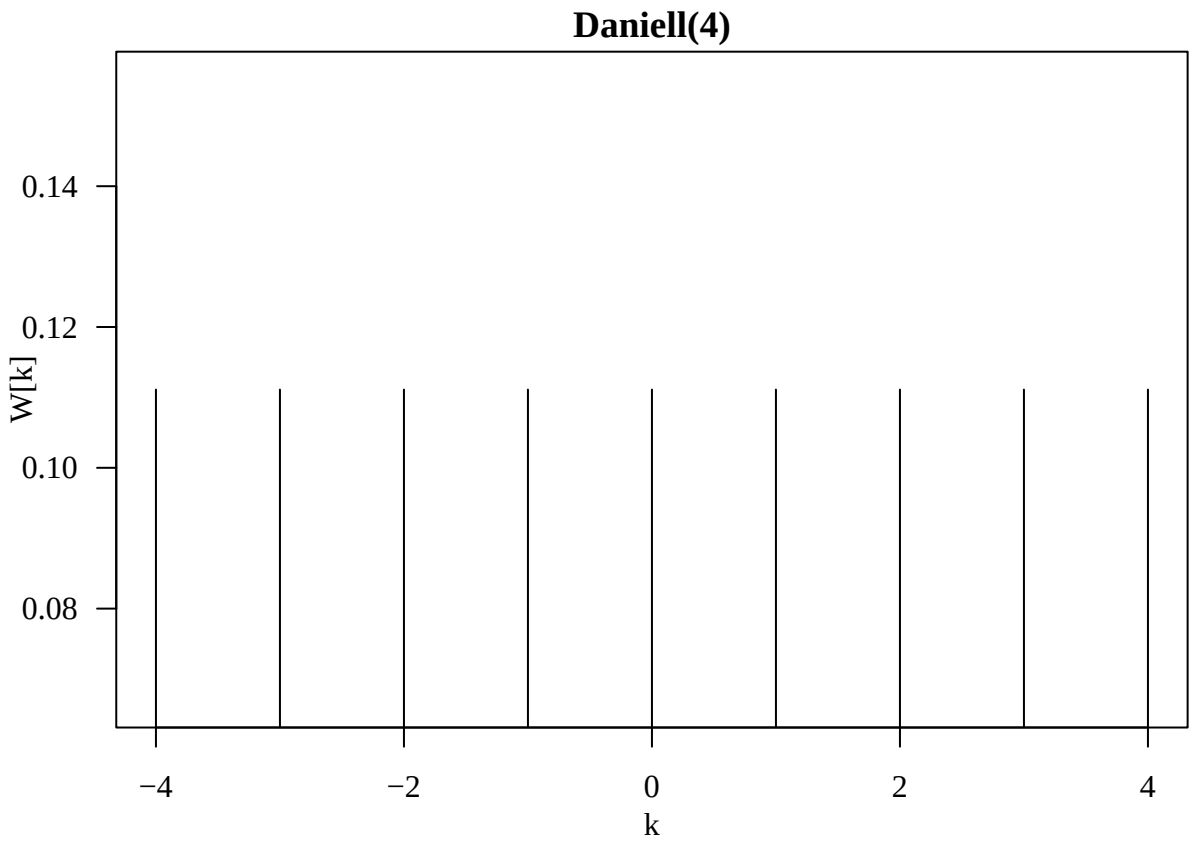
```
## [1]  0.2635573 38.4010800
```

Averaged periodogram

An approximate $100(1-\alpha)\%$ confidence interval for $f(\omega)$

$$\frac{2L\bar{f}(\omega)}{\chi^2_{2L}(1-\alpha/2)} \leq f(\omega) \leq \frac{2L\bar{f}(\omega)}{\chi^2_{2L}(\alpha/2)}$$

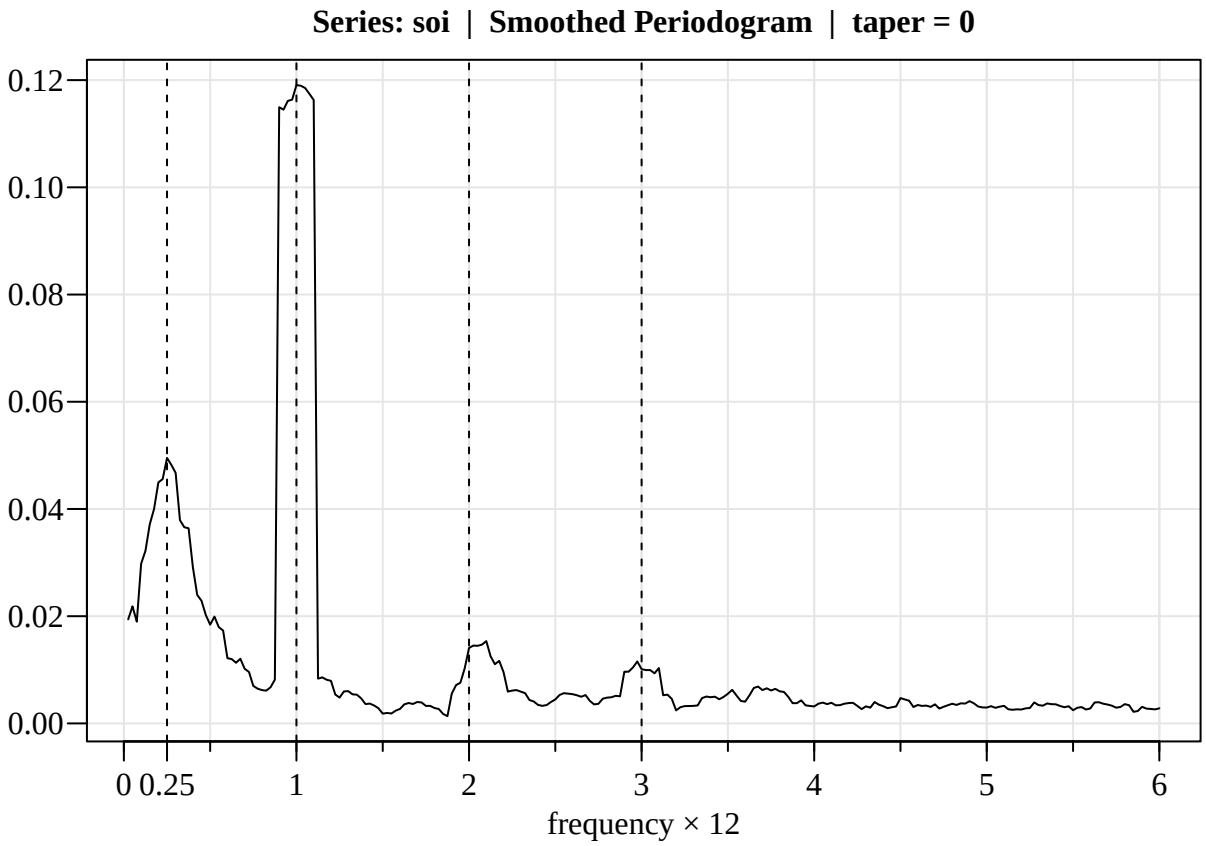
```
par(mgp = c(2, 1, 0), mar = c(3.5, 4, 1.4, 0.6), las = 1, family = "serif")
plot(kernel("daniell", 4))
```




```
par(mgp = c(3, 2, 0), mar = c(3.5, 4, 1.4, 0.6), las = 1, family = "serif")
soi.ave <- mvspec(soi, kernel('daniell', 4), ylab = "")
```

```
## Bandwidth: 0.225
## Degrees of Freedom: 16.99
```

```
abline(v = c(.25, 1, 2, 3), lty = 2)
axis(1, at = 0.25)
```




```
soi.ave$bandwidth
```

```
## [1] 0.225
```

```
(df <- soi.ave$df)
```

```
## [1] 16.9875
```

```
(U <- qchisq(.025, df))
```

```
## [1] 7.555916
```

```
(L <- qchisq(.975, df))
```

```
## [1] 30.17425
```

```
soi.ave$spec[10]
```

```
## [1] 0.04952026
```

```
soi.ave$spec[40]
```

```
## [1] 0.11908
```

```r
# intervals
c(df * soi.ave$spec[10] / L, df * soi.ave$spec[10] / U)
```

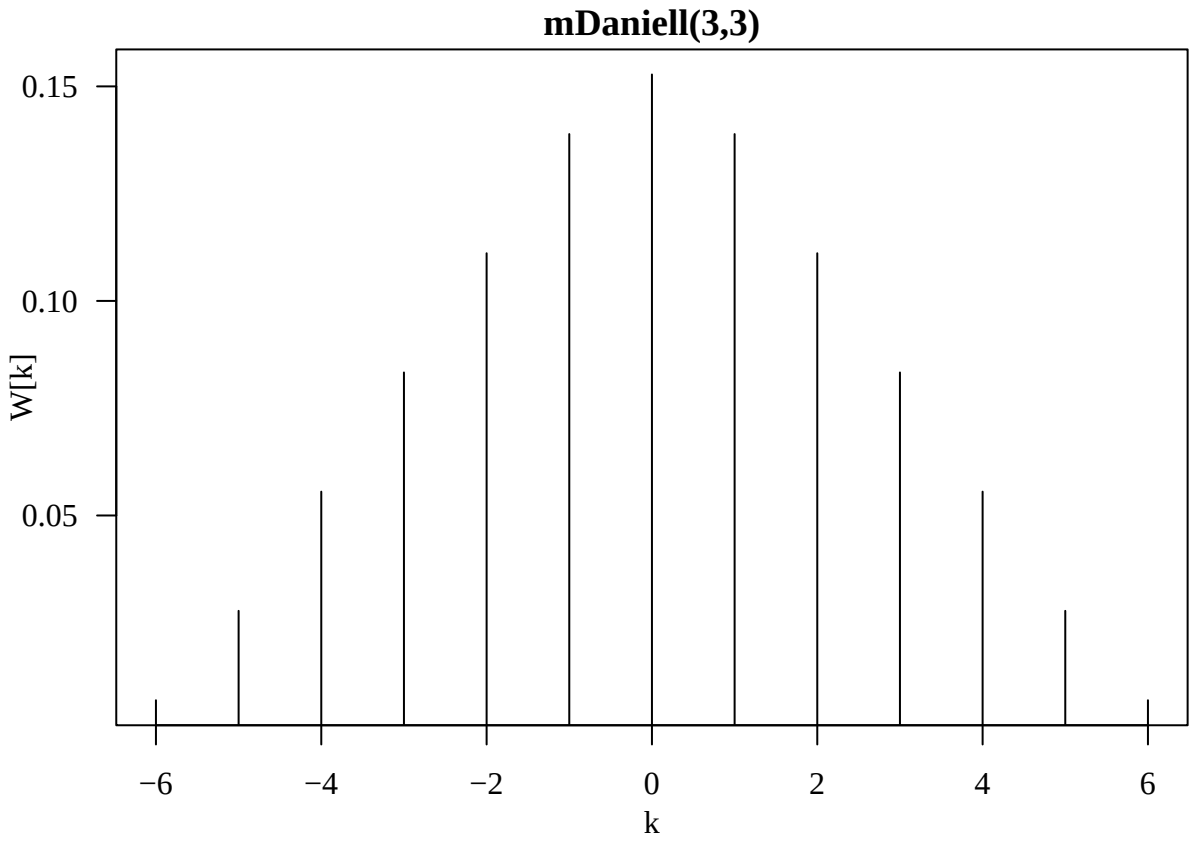
```
## [1] 0.02787891 0.11133335
```

```r
c(df * soi.ave$spec[40] / L, df * soi.ave$spec[40] / U)
```

```
## [1] 0.06703963 0.26772011
```

Smoothed periodogram

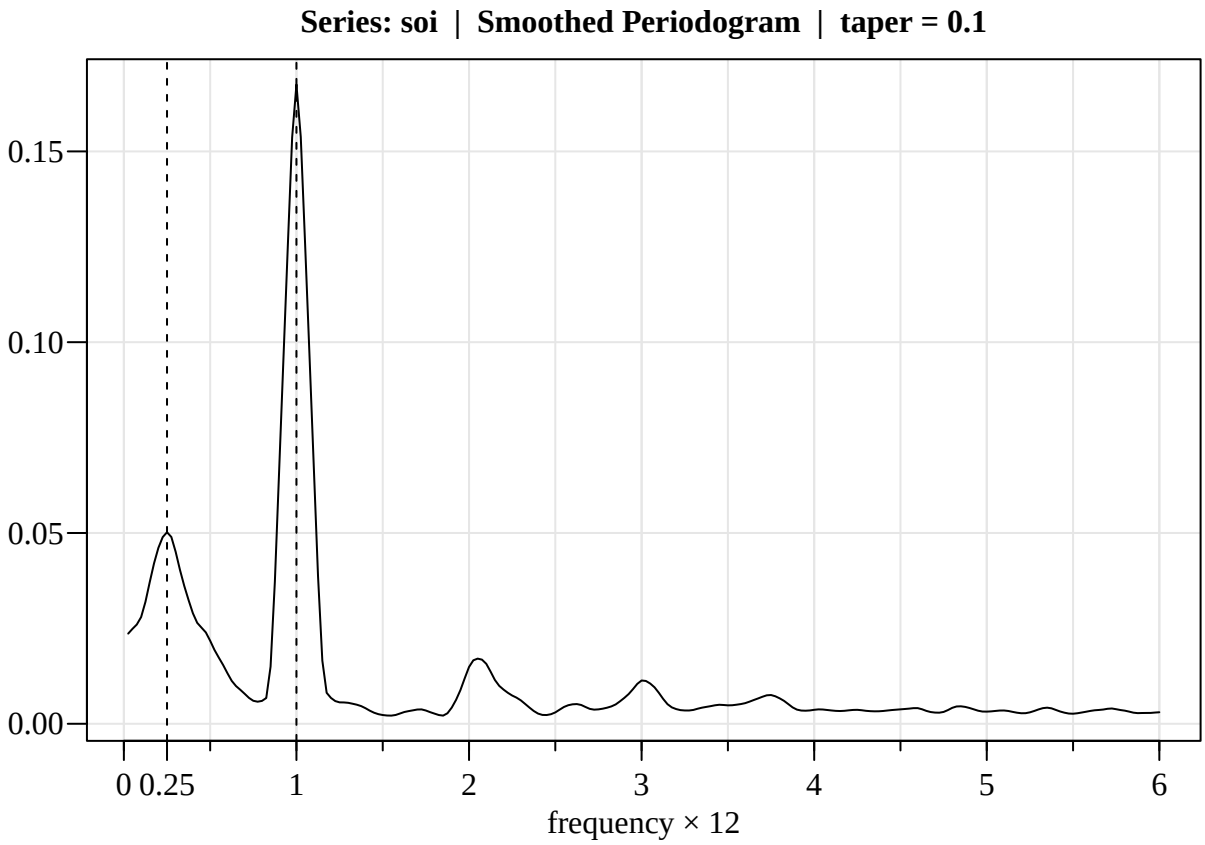
```r
par(mgp = c(2, 1, 0), mar = c(3.5, 4, 1.4, 0.6), las = 1, family = "serif")
plot(kernel("modified.daniell", c(3, 3)))
```




```r
k <- kernel("modified.daniell", c(3, 3))
par(mgp = c(3, 2, 0), mar = c(3.5, 4, 1.4, 0.6), las = 1, family = "serif")
soi.smo <- mvspec(soi, kernel = k, taper = .1, ylab = "")
```

```
## Bandwidth: 0.231
## Degrees of Freedom: 15.61
```

```
abline(v = c(.25, 1), lty = 2)
axis(1, at = 0.25)
```




```
soi.smo$bandwidth
```

```
## [1] 0.2308103
```

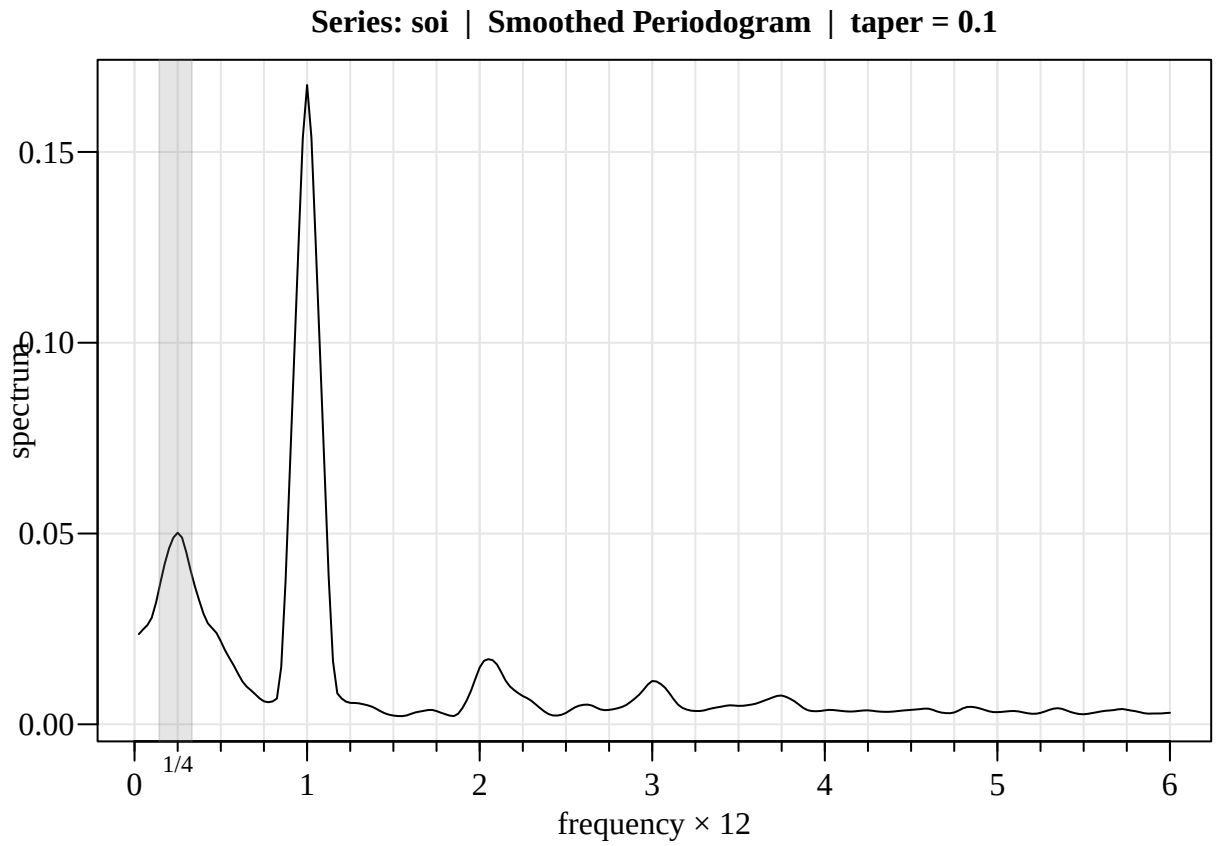
```
(df <- soi.smo$df)
```

```
## [1] 15.61029
```

```
soi.smo <- mvspec(soi, spans = c(7, 7), taper = .1, nxm = 4)
```

```
## Bandwidth: 0.231
## Degrees of Freedom: 15.61
```

```
rect(1/7, -1e5, 1/3, 1e5, density = NA, col = gray(.5, .2))
mtext("1/4", side = 1, line = 0, at = .25, cex = .75)
```




```
(U <- qchisq(.025, df))
```

```
## [1] 6.65473
```

```
(L <- qchisq(.975, df))
```

```
## [1] 28.31793
```

```
soi.smo$spec[10]
```

```
## [1] 0.05019866
```

```
soi.smo$spec[40]
```

```
## [1] 0.1675368
```

```
# intervals
c(df * soi.smo$spec[10] / L, df * soi.smo$spec[10] / U)
```

```
## [1] 0.02767206 0.11775316
```

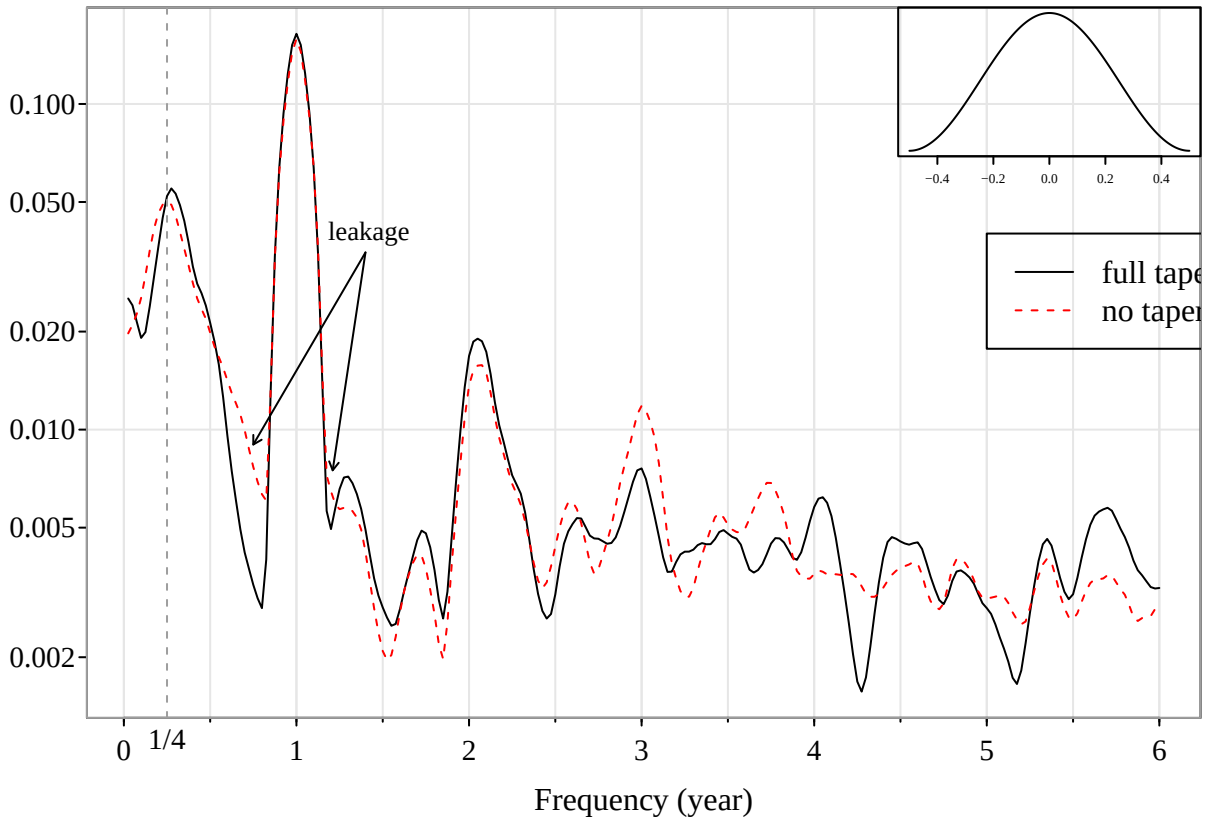
```
c(df * soi.smo$spec[40] / L, df * soi.smo$spec[40] / U)
```

```
## [1] 0.09235481 0.39299825
```

SOI tapering

```
par(mgp = c(3, 2, 0), mar = c(3.5, 4, 1.4, 0.6), las = 1, family = "serif")
s0 <- mvspec(soi, spans = c(7, 7), plot = FALSE)
s50 <- mvspec(soi, spans = c(7, 7), taper = .5, plot = FALSE)
tsplot(s50$freq, s50$spec, log = "y", type = "l", ylab = "", xlab = "Frequency (year)")
lines(s0$freq, s0$spec, lty = 2, col = "red")
abline(v = .25, lty = 2, col = 8)
mtext('1/4', side = 1, line = 0, at = .25, cex = .9)
legend(5, .04, legend = c('full taper', 'no taper'), lty = 1:2, col = c("black", "red"))

text(1.42, 0.04, 'leakage', cex = .8)
arrows(1.4, .035, .75, .009, length = 0.05, angle = 30)
arrows(1.4, .035, 1.21, .0075, length = 0.05, angle = 30)
par(fig = c(.65, 1, .65, 1),  new = TRUE, cex = .5,
    mgp = c(0, -.1, 0), tcl = -.2)
taper <- function(x) {.5 * (1 + cos(2 * pi * x))}
x <- seq(from = -.5, to = .5, by = 0.001)
plot(x, taper(x), type = "l",  lty = 1,  yaxt = 'n', ann = FALSE)
```



Parametric Spectral Estimation

We estimate a spectrum by first fitting a time-domain model. Here, `spec.ic()` selects an AR order using AIC and computes the spectrum of the fitted model. We compare that estimate with the known theoretical spectrum.

The three examples illustrate persistent dependence, an AR approximation to an ARMA process, and an oscillatory process with a narrow spectral peak. Frequencies are measured in cycles per observation.

Setup

We use the same sample size for all three examples. Setting the seed makes the simulated series reproducible.

```
library(TSA)
```

```
##
## Attaching package: 'TSA'
```

```
## The following objects are masked from 'package:stats':
##
##     acf, arima
```

```
## The following object is masked from 'package:utils':
##
##     tar
```

```
set.seed(12345)
n <- 200
phi <- 0.8
theta <- 0.5
phi1 <- 1.5
phi2 <- -0.95

y <- arima.sim(model = list(ar = phi), n = n)
y1 <- arima.sim(model = list(ar = phi, ma = theta), n = n)
y2 <- arima.sim(model = list(ar = c(phi1, phi2)), n = n)
```

AR(1): Persistence Appears as Low-Frequency Variation

For an AR(1) process with a positive coefficient, dependence decays gradually with lag. Its spectrum is largest near frequency zero, reflecting slow fluctuations rather than an oscillation with a fixed period.

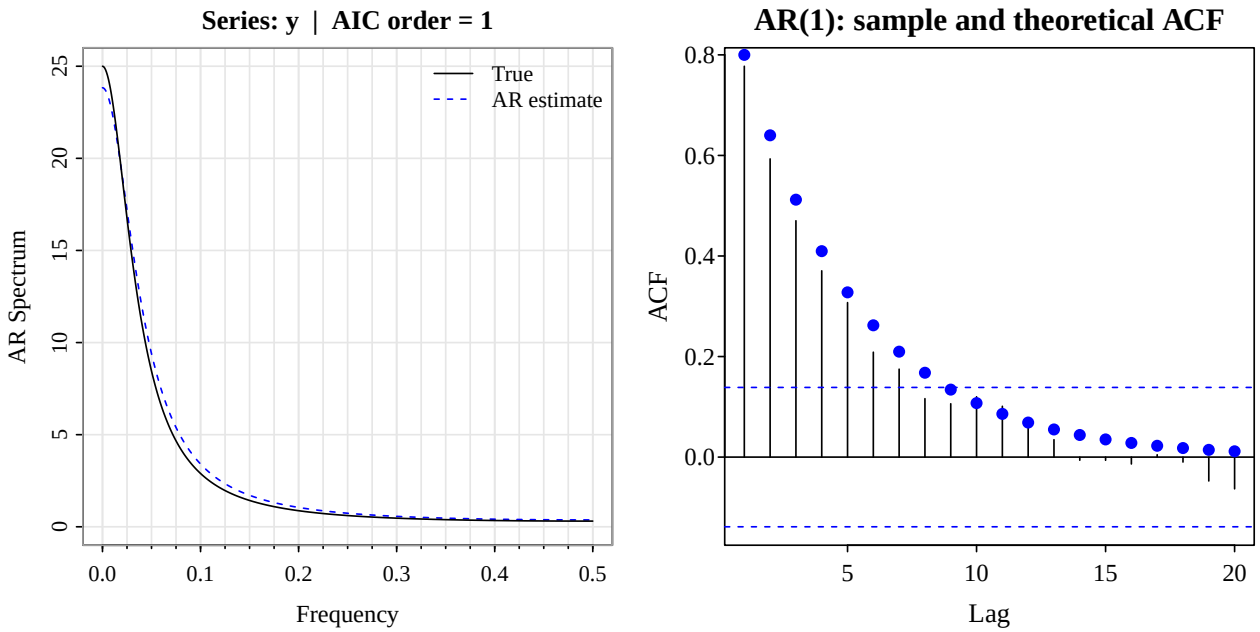
The left panel compares the fitted AR spectrum with the theoretical spectrum. The right panel compares the sample ACF with the theoretical ACF.

```
par(las = 1, mar = c(4, 4, 2, 0.6), mgp = c(3, 1, 0), mfrow = c(1, 2),
    family = "serif")

# AIC-selected AR spectrum.
sp_ar1 <- astsa::spec.ic(y, detrend = FALSE, col = "blue",
                         lwd = 1, nxm = 4, ylim = c(0, 25), lty = 2)
freq <- sp_ar1[[2]][, 1]
```

```r
# Overlay the theoretical spectrum.
lines(freq, TSA::ARMAspec(model = list(ar = phi), freq = freq, plot = FALSE)$spec,
      col = "black")
legend("topright", c("True", "AR estimate"),
       col = c("black", "blue"), lty = c(1, 2), bty = "n", cex = 0.8)

acf(y, lag.max = 20, main = "AR(1): sample and theoretical ACF")
acf_true <- ARMAacf(ar = phi, lag.max = 20)
points(1:20, acf_true[-1], col = "blue", pch = 16)
```



Read the two panels together: the gradually decaying ACF and the concentration of spectral density near zero describe the same persistence in different domains. The selected AR order may vary across simulated samples.

ARMA(1,1): Using an AR Model as an Approximation

The second series includes both autoregressive and moving-average dependence. Nevertheless, we again fit candidate **AR models** using `spec.ic()`.

This illustrates an important distinction: our aim is to approximate the spectral shape, not necessarily to recover the exact data-generating model. A higher-order AR model can approximate the dependence of a stationary, invertible ARMA process.

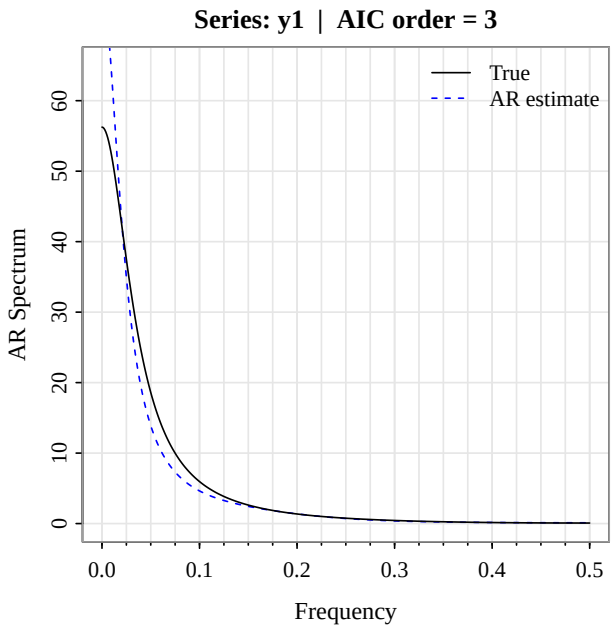
```r
par(las = 1, mar = c(4, 4, 2, 0.6), mgp = c(3, 1, 0), mfrow = c(1, 2),
    family = "serif")

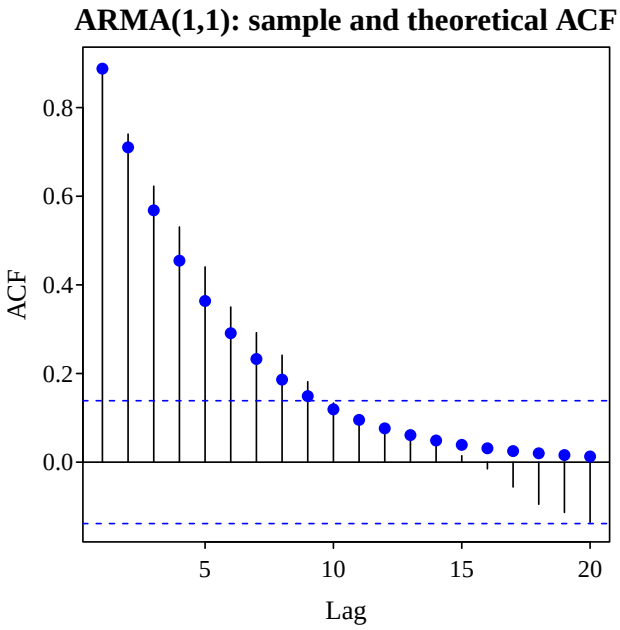
# Fit an AR approximation to the ARMA-generated series.
sp_arma11 <- astsa::spec.ic(y1, detrend = FALSE, col = "blue",
                            lwd = 1, nxm = 4, ylim = c(0, 65), lty = 2)
freq <- sp_arma11[[2]][, 1]

lines(freq, TSA::ARMAspec(model = list(ar = phi, ma = theta),
```

```
                          freq = freq, plot = FALSE)$spec, col = "black")
legend("topright", c("True", "AR estimate"),
       col = c("black", "blue"), lty = c(1, 2), bty = "n", cex = 0.8)

# Use y1 here: this is the ARMA series.
acf(y1, lag.max = 20, main = "ARMA(1,1): sample and theoretical ACF")
acf_true <- ARMAacf(ar = phi, ma = theta, lag.max = 20)
points(1:20, acf_true[-1], col = "blue", pch = 16)
```





Compare the fitted and theoretical spectra, especially their low-frequency concentration. Agreement indicates that the AR approximation captures the main spectral structure, even though the generating model contains a moving-average term.

AR(2): Resolving a Narrow Spectral Peak

The final process has damped oscillatory dependence, producing a pronounced peak away from frequency zero. We compare the AR-based estimate with a nonparametric Daniell estimate that averages seven neighboring periodogram ordinates (`l = 3`).

This example highlights the tradeoff between smoothing and spectral resolution.

```
# Return to a single-panel layout.
par(las = 1, mar = c(4, 4, 2, 0.6), mgp = c(3, 1, 0), mfrow = c(1, 1),
    family = "serif")

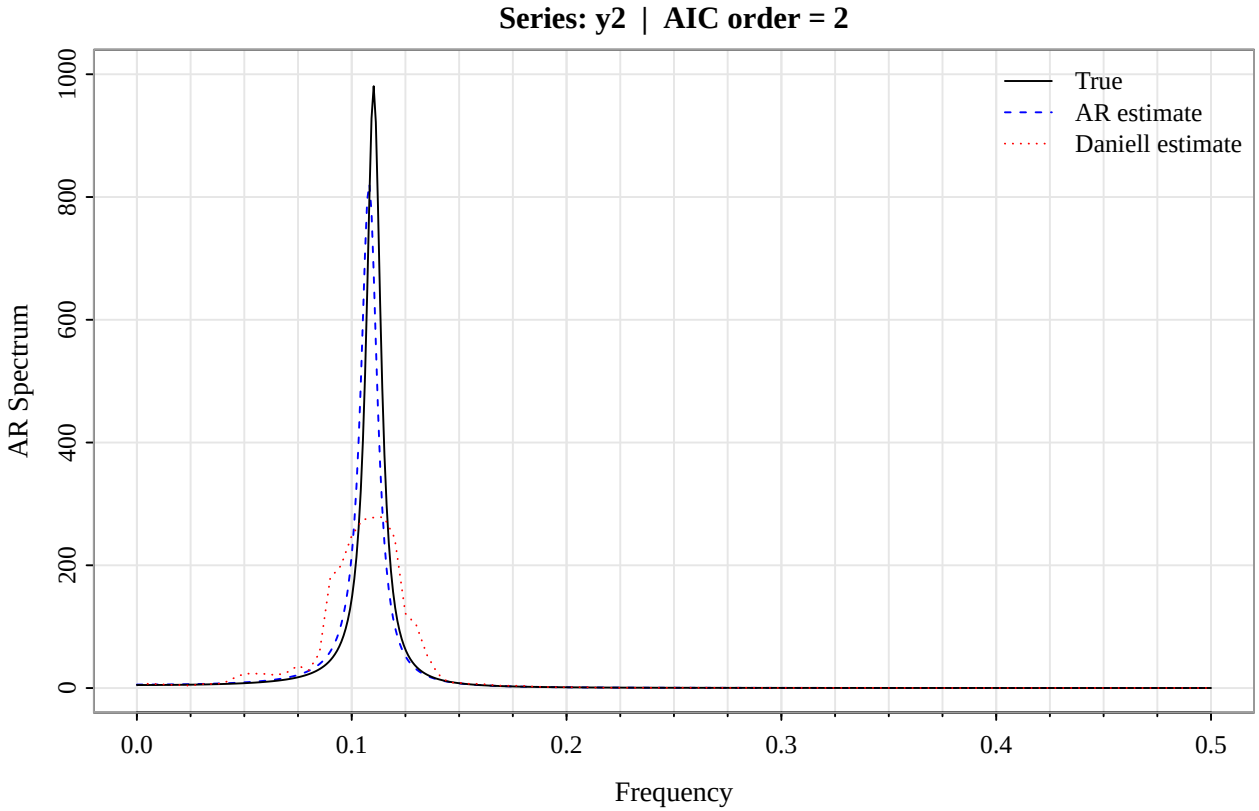
sp_ar2 <- astsa::spec.ic(y2, detrend = FALSE, col = "blue",
                         lwd = 1, nxm = 4, ylim = c(0, 1000), lty = 2)
freq <- sp_ar2[[2]][, 1]

lines(freq, TSA::ARMAspec(model = list(ar = c(phi1, phi2)),
                          freq = freq, plot = FALSE)$spec, col = "black")

# Nonparametric comparison: same data, no taper or detrending.
```

```
sp_nonpar <- astsa::mvspec(y2, kernel = stats::kernel("daniell", 3),
                           taper = 0, detrend = FALSE, plot = FALSE)
lines(sp_nonpar$freq, sp_nonpar$spec, col = "red", lty = 3)

legend("topright", c("True", "AR estimate", "Daniell estimate"),
       col = c("black", "blue", "red"), lty = c(1, 2, 3), bty = "n", cex = 0.85)
```



Compare **peak location, width, and height separately**. The Daniell smoother tends to broaden and lower a narrow peak because it averages across nearby frequencies. A suitable AR model can preserve a sharper peak, although its estimated height and location still vary across samples.

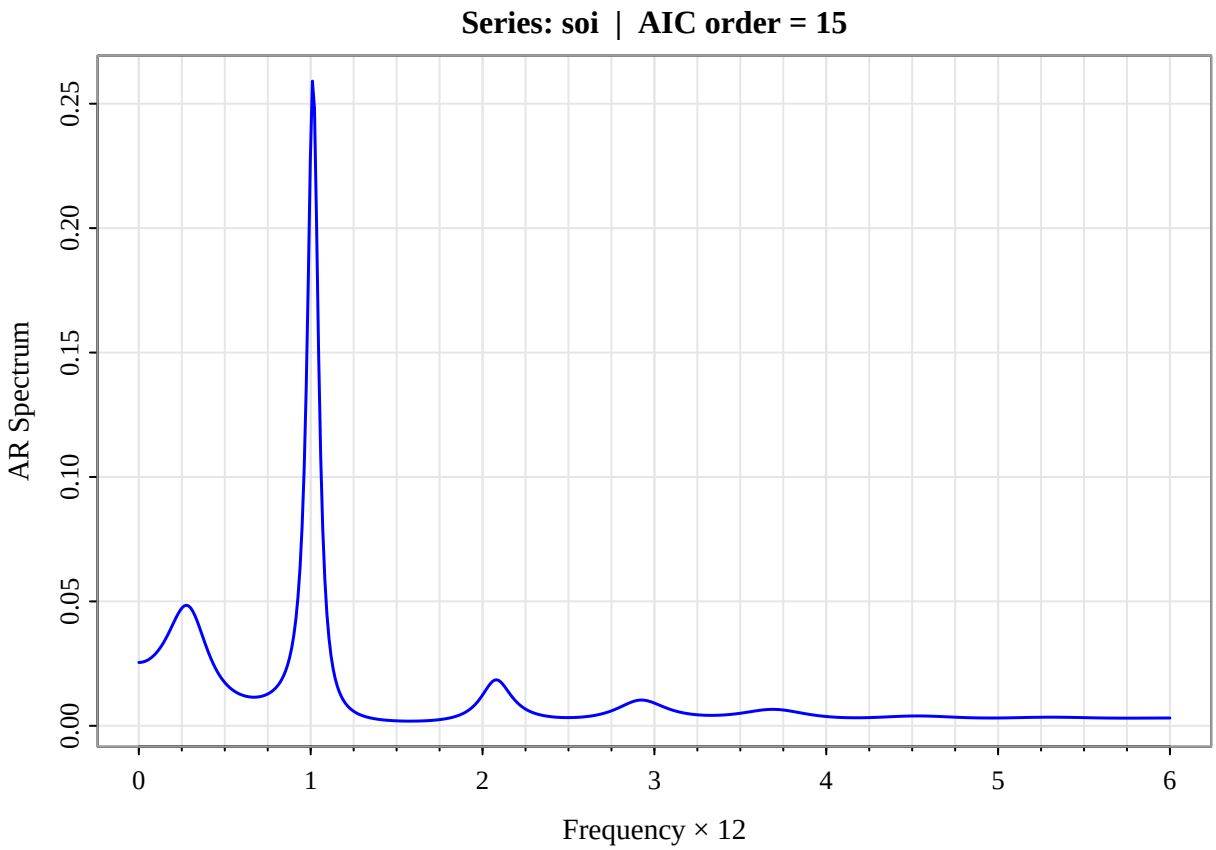
The broader lesson is that a sharper curve is not automatically a better estimate. Parametric resolution comes from model assumptions; nonparametric stability comes from averaging. Comparing the two helps us judge which spectral features are robust.

SOI Example: Choosing the AR Order

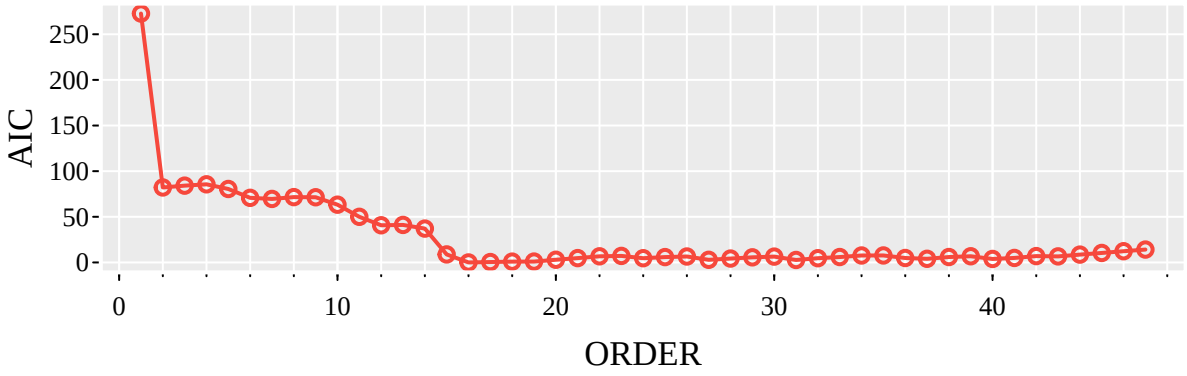
We now estimate the SOI spectrum using an AR model. Unlike the simulated examples, the true spectrum is unknown. We therefore examine both the fitted spectral features and the information criteria used to choose model complexity.

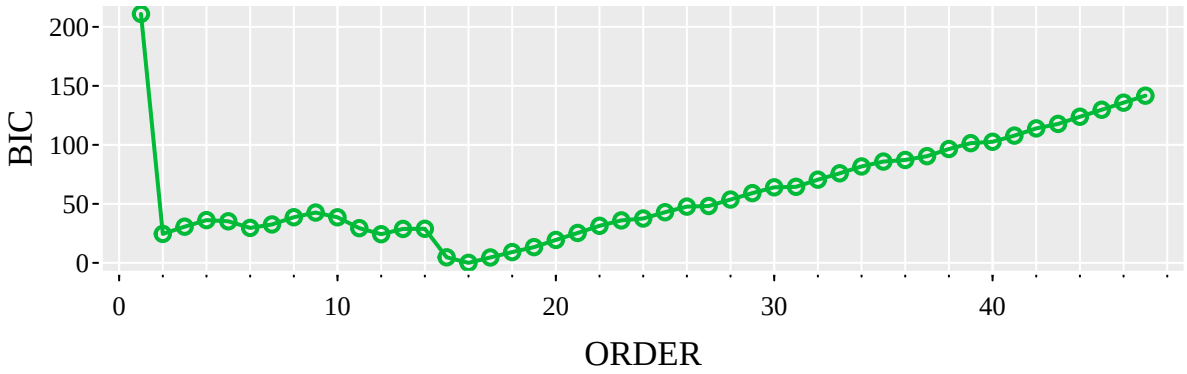
The code below removes a fitted linear trend before estimating the spectrum. **Detrending does not remove a recurring monthly mean**; seasonal adjustment is a separate modeling step.

```
par(las = 1, mar = c(4, 4, 2, 0.6), mgp = c(3, 1, 0), family = "serif")
u <- spec.ic(soi, detrend = TRUE, col = "blue", lwd = 1.5, nxm = 4)
```




```
# plot AIC and BIC
tsplot(1:47, u[[1]][, 2:3], type = 'o', col = 2:3, xlab = 'ORDER', nxm = 5, lwd = 2, gg = TRUE)
```





Because `soi` is a monthly `ts` object, the frequency axis is in **cycles per year**. A peak near frequency 1 represents annual variation; a feature near 0.25 represents variation over roughly four years.

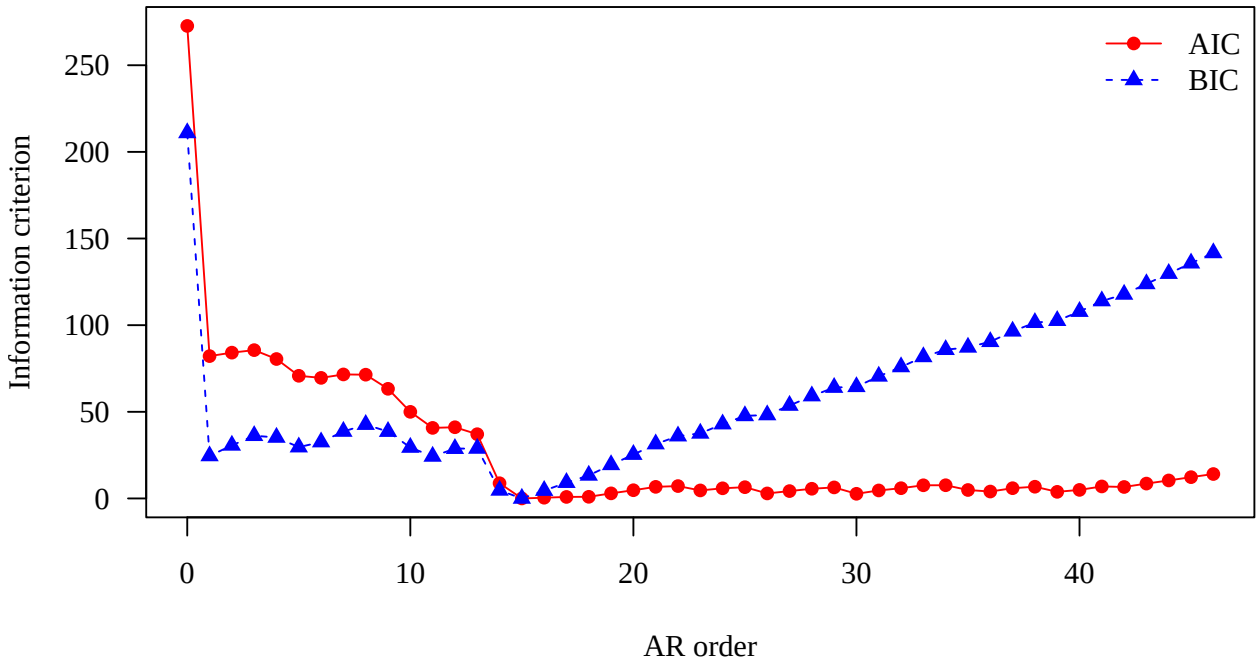
Read the fitted spectrum as a model-based summary of these time scales. In particular, a narrow annual peak does not establish that the detrended series has a constant mean throughout the calendar year.

How Strongly Do the Criteria Favor the Selected Order? The spectral curve depends on the AR order. To understand that choice, we plot AIC and BIC across the candidate models. Smaller values indicate a preferred balance between fit and complexity.

```
par(las = 1, mar = c(4, 4, 2, 0.6),
    mgp = c(3, 1, 0), mfrow = c(1, 1), family = "serif")

# Extract the evaluated orders and their information criteria.
ic <- u[[1]]
orders <- ic[, 1]
criteria <- ic[, 2:3, drop = FALSE]

matplot(orders, criteria, type = "o", col = c("red", "blue"), lty = c(1, 2), pch = c(16, 17),
        xlab = "AR order", ylab = "Information criterion")
legend("topright", c("AIC", "BIC"),
       col = c("red", "blue"), lty = c(1, 2), pch = c(16, 17), bty = "n")
```




```
# Report the minimizing orders rather than reading them by eye.
data.frame(Criterion = c("AIC", "BIC"),
           Selected_order = c(orders[which.min(criteria[, 1])],
                              orders[which.min(criteria[, 2])]))
```

```
##   Criterion Selected_order
## 1       AIC             15
## 2       BIC             15
```

BIC penalizes additional parameters more strongly than AIC and therefore often selects a smaller order.

A clear minimum supports a particular order more strongly than a broad, nearly flat region. If several orders have similar criterion values, compare their fitted spectra: do the annual and multiyear features persist, or do narrow peaks appear only at particular orders?

Main message: information criteria help choose the complexity of the AR approximation. They do not establish physical cycle lengths or guarantee that every fitted spectral peak is real. Interpret the selected spectrum alongside the nonparametric estimates and the chosen trend/seasonal mean model.

Lagged regression

In this section, we study the relationship between the Southern Oscillation Index (SOI) and fish recruitment using lagged regression and frequency-domain tools.

We'll go through:

1. Visualizing the SOI and recruitment series, and their cross-correlation
2. Building and interpreting a transfer function (lagged regression) model in the time domain
3. Estimating the cross-spectrum and coherence in the frequency domain
4. Fitting a lagged regression model via `LagReg()` and interpreting the final equation

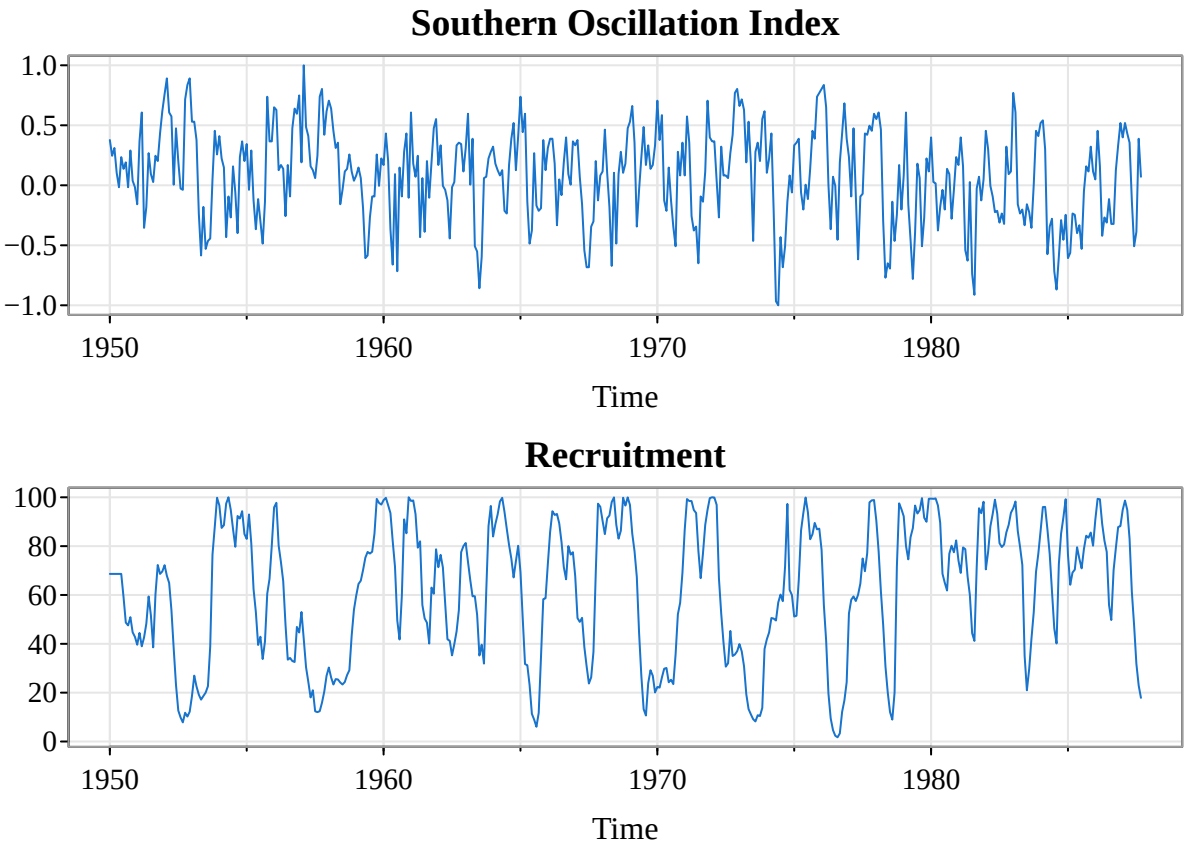
SOI and Recruitment Time Series

We start by plotting the time series and examining their autocorrelation structures.

```
par(mfrow = c(2, 1), las = 1, family = "serif")

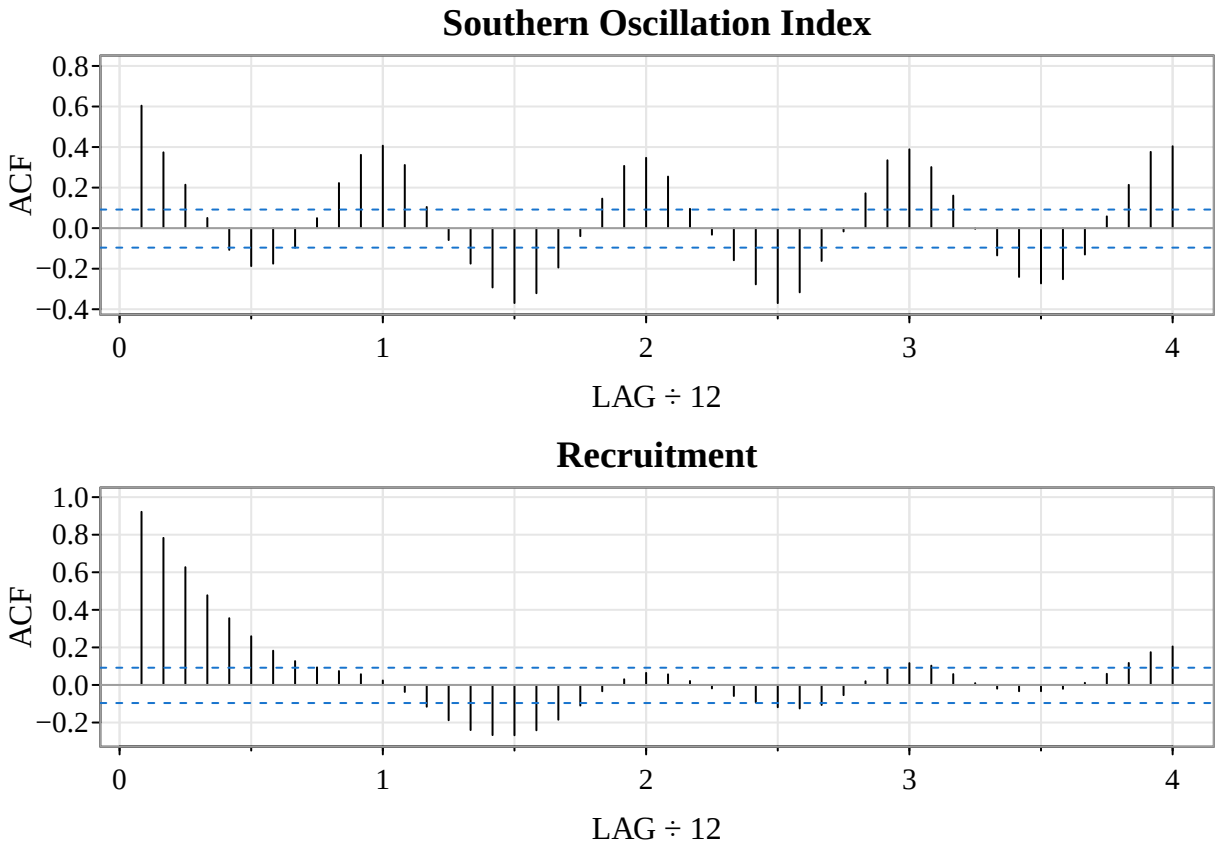
# Plot 1: Southern Oscillation Index (SOI)
tsplot(soi, col = 4, ylab = "", main = "Southern Oscillation Index")

# Plot 2: Recruitment time series
tsplot(rec, col = 4, ylab = "", main = "Recruitment")
```



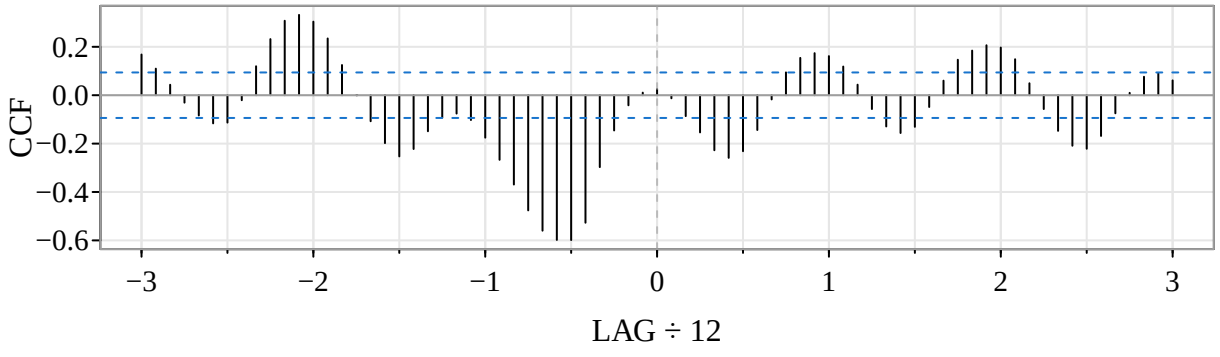

```
# Next, we examine autocorrelation and cross-correlation structures
# ACF of SOI
invisible(acf1(soi, main = "Southern Oscillation Index"))

# ACF of Recruitment
invisible(acf1(rec, main = "Recruitment"))
```



The cross-correlation function (CCF) shows how the Southern Oscillation Index (SOI) and fish recruitment are associated at different time lags. If we define the CCF at lag (h) as the correlation between SOI_{t+h} and Recruitment$t$, a negative lag means that SOI leads recruitment by (h) time steps. For example, with monthly data, a strong correlation at lag (-6) indicates an association between SOI and recruitment six months later.

```
par(mfrow = c(1, 1), las = 1, family = "serif")
ccf2(soi, rec, main = "SOI vs. Recruitment")
```



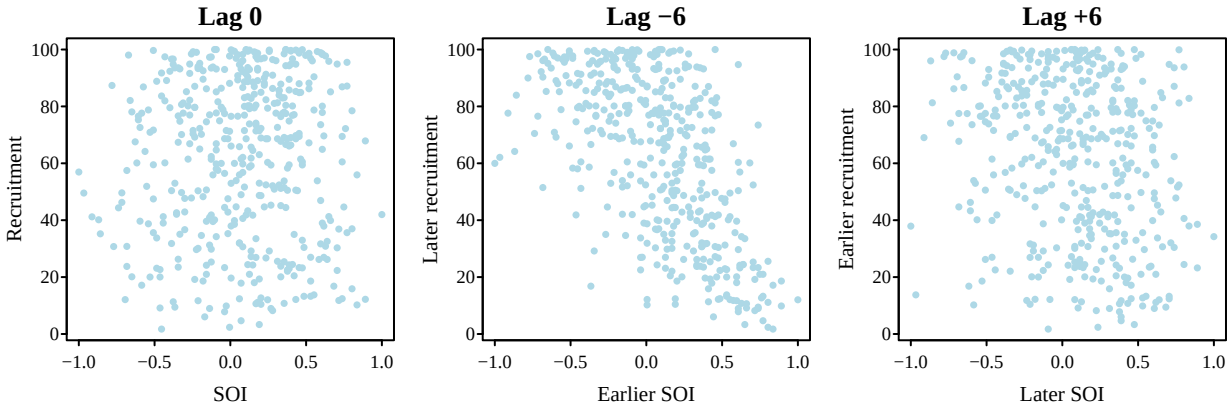

```
n <- length(soi)
early <- seq_len(n - 6)
late <- early + 6

par(mfrow = c(1, 3))
```

```r
# Lag 0: same-month pairs
plot(soi, rec, pch = 16, cex = 0.75, col = "lightblue",
     xlab = "SOI", ylab = "Recruitment", main = "Lag 0")

# Lag -6: SOI leads recruitment
plot(soi[early], rec[late], pch = 16, cex = 0.75, col = "lightblue",
     xlab = "Earlier SOI", ylab = "Later recruitment", main = "Lag -6")

# Lag +6: recruitment leads SOI
plot(soi[late], rec[early], pch = 16, cex = 0.75, col = "lightblue",
     xlab = "Later SOI", ylab = "Earlier recruitment", main = "Lag +6")
```




```r
par(mfrow = c(1, 1))

round(c("Lag 0" = cor(soi, rec), "Lag -6" = cor(soi[early], rec[late]),
        "Lag +6" = cor(soi[late], rec[early])), 3)
```

```
##  Lag 0 Lag -6 Lag +6
##  0.025 -0.602 -0.234
```

The raw CCF can be misleading when the input series is autocorrelated. In this R example, (Y_t) depends only on X_{t-6}, but the persistence in X_t produces sizable cross-correlations at neighboring lags. This can make a single six-period delay look like a response spread over several lags, motivating the use of prewhitening.

```r
set.seed(8090)

n <- 500
ell <- 6
phi <- 0.9

# Persistent input and a response driven only by lag 6
x_full <- as.numeric(arima.sim(list(ar = phi), n = n + ell))
x <- x_full[ell + seq_len(n)]
y <- x_full[seq_len(n)] + rnorm(n)

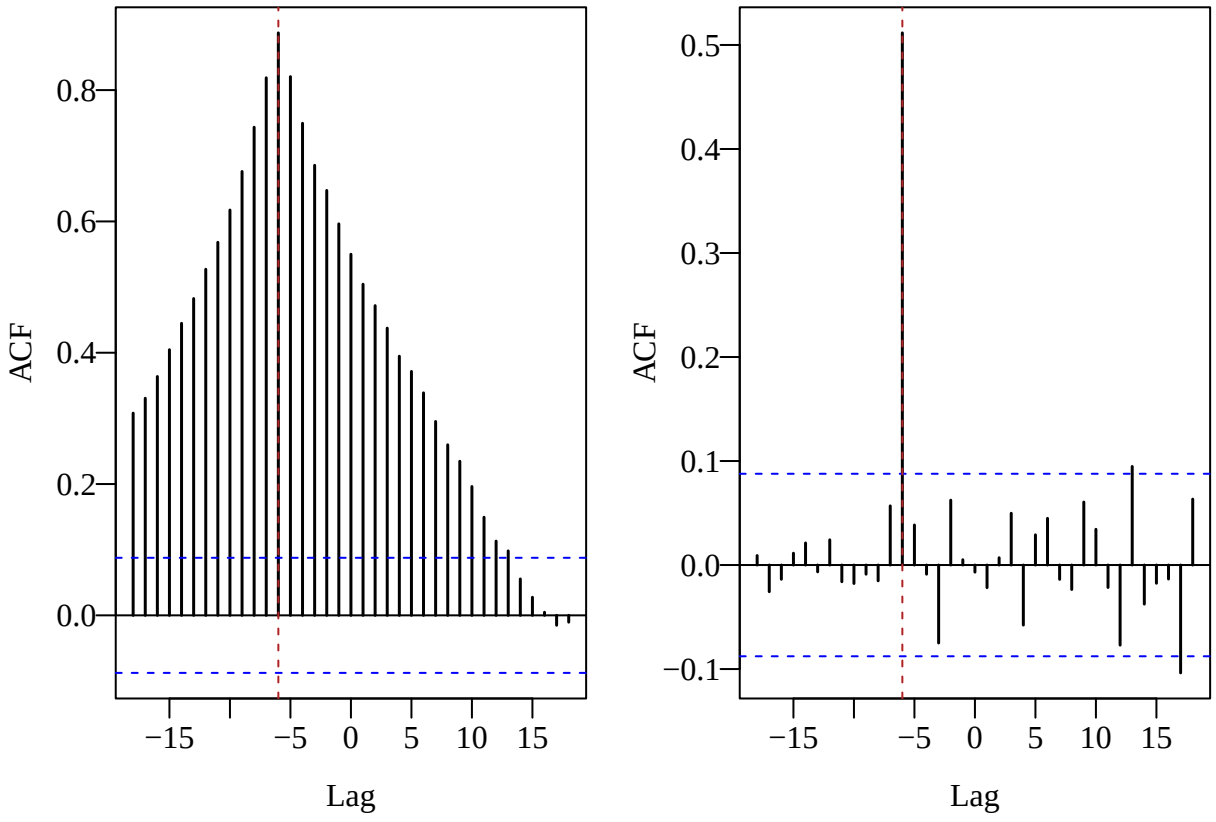
# Estimate the AR(1) filter from X and apply it to both series
phi_hat <- unname(coef(arima(x, order = c(1, 0, 0)))["ar1"])
x_white <- stats::filter(x, c(1, -phi_hat), sides = 1)
```

```r
y_filtered <- stats::filter(y, c(1, -phi_hat), sides = 1)

old_par <- par(mfrow = c(1, 2), mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 0.5, 0),
               las = 1, family = "serif")

ccf(x, y, lag.max = 18, main = "Raw CCF", lwd = 1.5)
abline(v = -ell, col = "firebrick", lty = 2)

ccf(x_white[-1], y_filtered[-1], lag.max = 18, lwd = 1.5,
    main = "CCF After Prewhitening")
abline(v = -ell, col = "firebrick", lty = 2)
```




```r
par(old_par)
```

In the model $Y_t = \beta X_{t-\ell} + V_t$, with V_t uncorrelated with X at all lags, the cross-covariance under R's convention is

$$\mathrm{Cov}(X_{t+h}, Y_t) = \beta\, \gamma_X(h + \ell).$$

So the raw CCF reflects a shifted version of the input's autocorrelation. The peak can still point to the right delay, but the neighboring bars can make a single-lag effect look like a distributed response. Applying the same filter to both series reduces the input's persistence while preserving the delay, making the feature at -6 easier to isolate.

Time-Domain Approach

We begin with a simple example of identifying an input–output relationship in the time domain. Our goal is to investigate whether earlier values of SOI help explain recruitment, while accounting for the persistence

within each series.

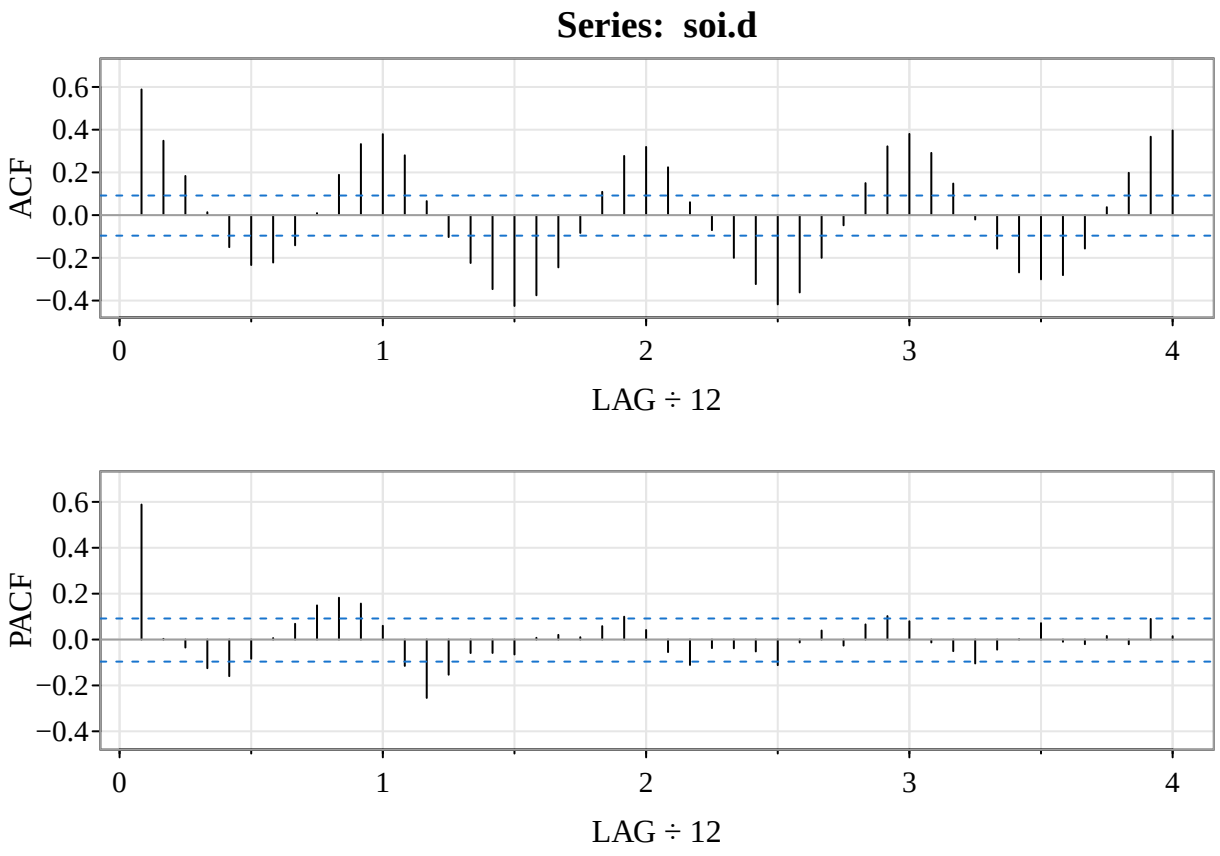
The raw CCF can be difficult to interpret because it combines the input's autocorrelation with the underlying input–output response. Prewhitening helps separate these features: we estimate a filter that makes SOI approximately white noise, apply the same filter to recruitment, and examine the cross-correlation between the transformed series.

For illustration, we use the following steps:

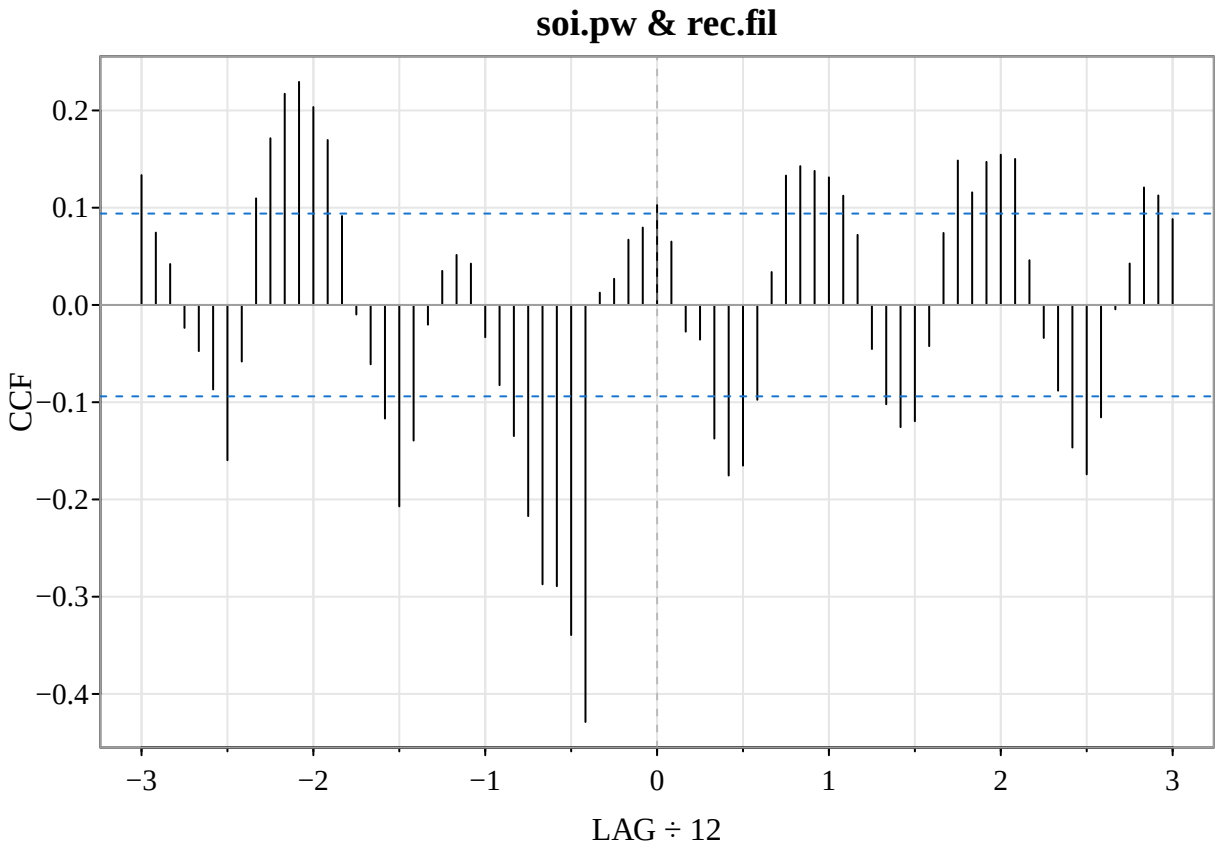
1. Remove a linear trend from SOI.
2. Fit an AR(1) model to detrended SOI.
3. Apply the same AR(1) filter to recruitment.
4. Examine the filtered CCF for candidate input–output lags.

These are illustrative modeling choices. Detrending should be supported by the data, and an AR(1) model is appropriate for prewhitening only if its residuals are approximately uncorrelated. The filtered CCF suggests candidate response lags; it does not determine the final model or establish causality.

```
par(las = 1, family = "serif")
# Step 1: Detrend SOI
soi.d <- resid(lm(soi ~ time(soi), na.action = NULL))
# Examine the ACF and PACF of detrended SOI
invisible(acf2(soi.d))
```



```r
# Step 2: Fit AR(1) model to detrended SOI (prewhitening)
fit <- arima(soi.d, order = c(1, 0, 0))
# Extract the AR(1) coefficient (phi)
ar1 <- as.numeric(coef(fit)[1])
soi.pw <- resid(fit)
# Step 3: Apply the same AR(1) filter to the recruitment
rec.fil <- filter(rec, filter = c(1, -ar1), sides = 1)
# Step 4: Cross-correlation between prewhitened SOI and filtered rec
ccf2(soi.pw, rec.fil)
```



After fitting the input model, inspect the ACF of its residuals. If substantial autocorrelation remains, the input has not been adequately prewhitened. Consider a higher-order AR model or a parsimonious (S)ARMA model, then apply that model's whitening filter to both series.

Look for a coherent pattern across neighboring lags rather than selecting every bar that crosses a reference line. An isolated feature may suggest a concentrated response, whereas a sequence of features may suggest a distributed response. These patterns guide the models we consider next.

Building a Simple Dynamic Regression Model

Next, we illustrate a dynamic regression model that includes recruitment at the previous month and detrended SOI five months earlier:

$$\text{rec}_t = \alpha + a_1 \text{rec}_{t-1} + \beta \text{SOI}_{t-5}^{(detrend)} + u_t.$$

The lagged recruitment term represents persistence in the output, while the lagged SOI term represents an additional association with earlier input conditions. Here, lag 5 is an illustrative candidate to investigate, rather than a lag we assume is optimal.

We first fit the model by OLS and inspect its residuals. Remaining autocorrelation suggests that the current specification does not fully describe the dynamics. Possible explanations include omitted response lags, seasonal structure, or a more complicated disturbance process.

This is a simple starting model. A satisfactory analysis should assess both the input–output response and the remaining noise structure, rather than judging the model only by how closely its fitted values follow the observations.

```
par(las = 1, family = "serif")

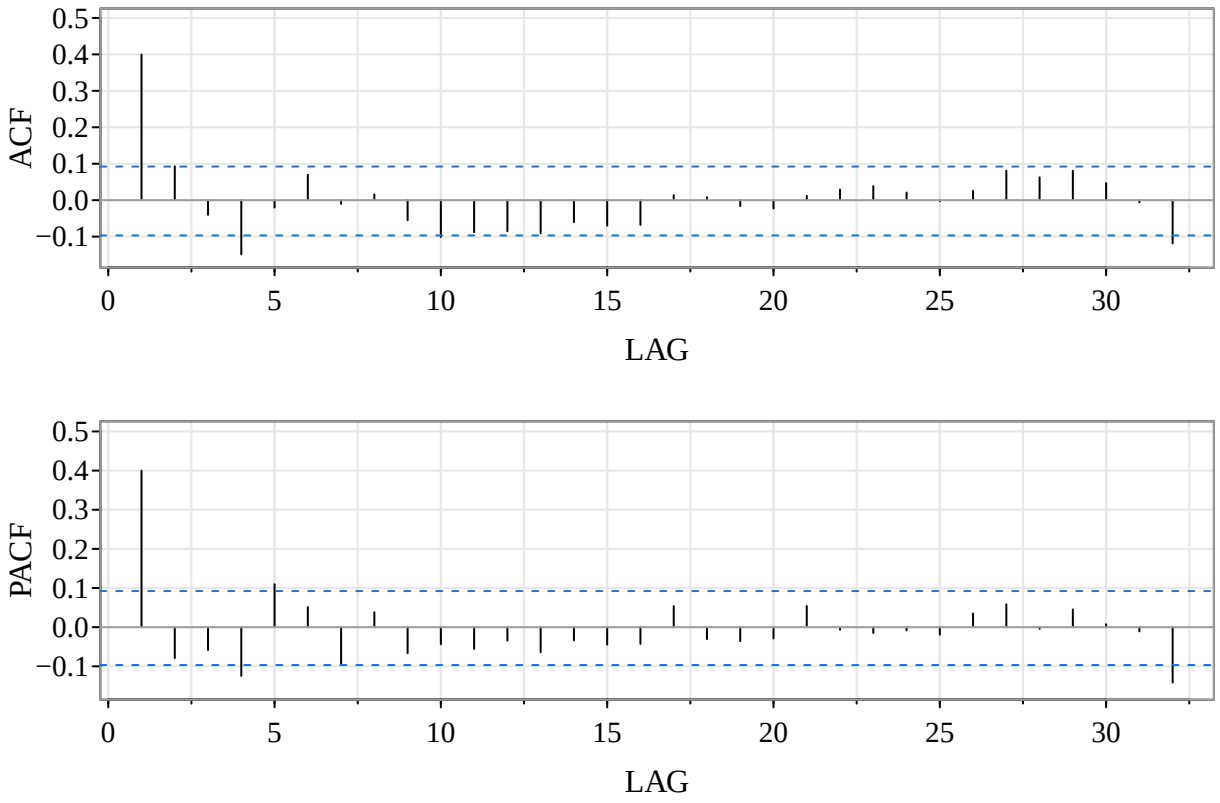
# Preserve the time index of detrended SOI.
soi.d <- ts(as.numeric(soi.d), start = start(soi), frequency = frequency(soi))

# Align recruitment with its own lag and lagged SOI.
fish <- ts.intersect(REC = rec, RL1 = stats::lag(rec, -1), SL5 = stats::lag(soi.d, -5))

# Model A: ARX model
# REC_t = alpha + a * REC_{t-1} + b * SOI.d_{t-5} + e_t
# Lagged recruitment represents persistence in the output.
fit.arx <- lm(REC ~ RL1 + SL5, data = as.data.frame(fish))
summary(fit.arx)
```

```
##
## Call:
## lm(formula = REC ~ RL1 + SL5, data = as.data.frame(fish))
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -51.262  -3.961   0.185   4.359  29.523
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   8.89711    0.92485    9.62   <2e-16 ***
## RL1           0.85561    0.01361   62.85   <2e-16 ***
## SL5         -20.37707    1.01631  -20.05   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.793 on 445 degrees of freedom
## Multiple R-squared:  0.9236, Adjusted R-squared:  0.9232
## F-statistic:  2689 on 2 and 445 DF,  p-value: < 2.2e-16
```

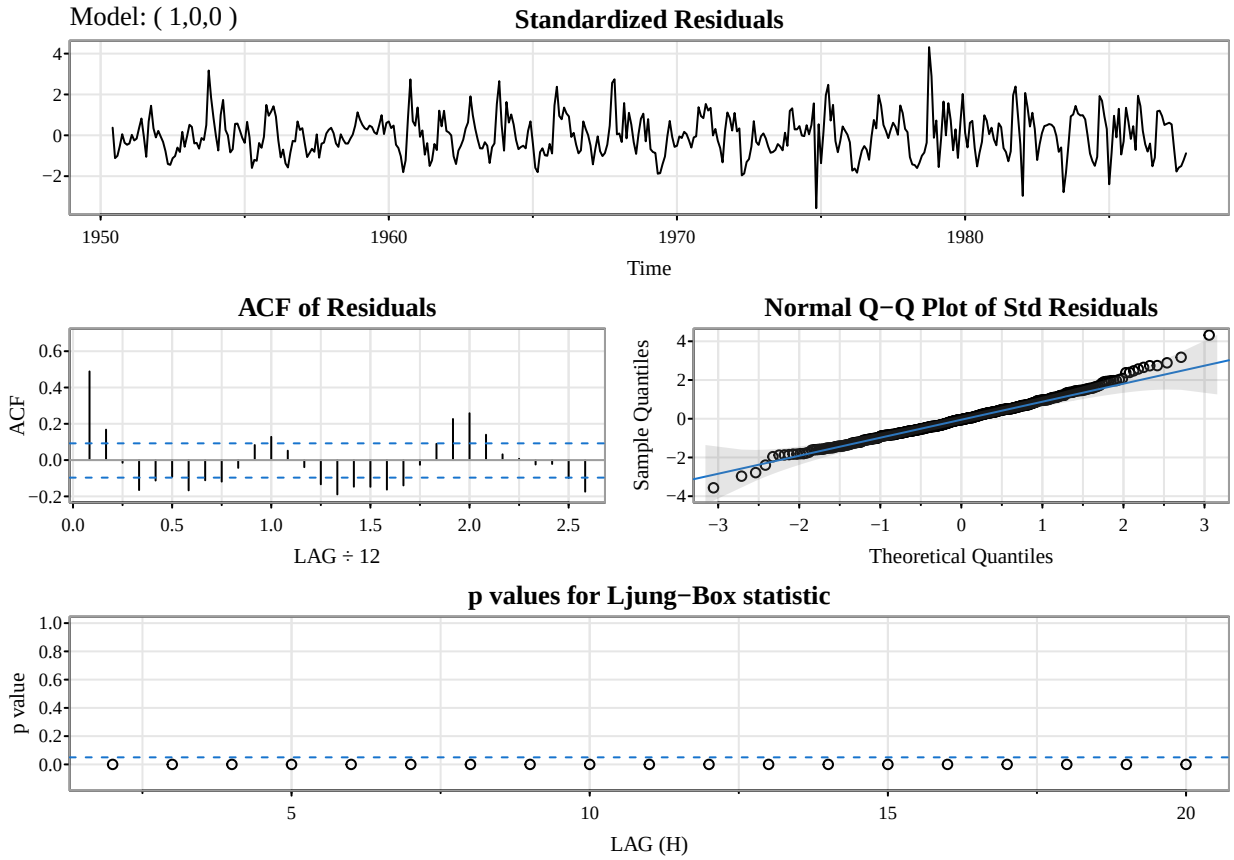
```
# Check whether the residuals are approximately uncorrelated.
invisible(acf2(residuals(fit.arx)))
```




```
# Model B: Lagged-SOI regression with AR(1) errors
# REC_t = alpha + b * SOI.d_{t-5} + u_t
# u_t = rho * u_{t-1} + e_t
# This alternative represents persistence through the errors.
fit.regAR <- sarima(fish[, "REC"], 1, 0, 0, xreg = fish[, "SL5", drop = FALSE])
```

```
## initial  value 3.195708
## iter   2 value 2.484013
## iter   3 value 2.438025
## iter   4 value 2.323740
## iter   5 value 2.262529
## iter   6 value 2.260715
## iter   7 value 2.260536
## iter   8 value 2.260493
## iter   9 value 2.260492
## iter  10 value 2.260491
## iter  11 value 2.260490
## iter  12 value 2.260475
## iter  13 value 2.260460
## iter  14 value 2.260446
## iter  15 value 2.260442
## iter  16 value 2.260442
## iter  17 value 2.260442
## iter  18 value 2.260442
## iter  18 value 2.260442
## iter  18 value 2.260442
```

```
## final  value 2.260442
## converged
## initial  value 2.261745
## iter   2 value 2.261742
## iter   3 value 2.261732
## iter   4 value 2.261732
## iter   5 value 2.261729
## iter   6 value 2.261726
## iter   7 value 2.261722
## iter   8 value 2.261721
## iter   9 value 2.261721
## iter  10 value 2.261721
## iter  10 value 2.261721
## iter  10 value 2.261721
## final  value 2.261721
## converged
## <><><><><><><><><><><><><><>
##
## Coefficients:
##           Estimate     SE  t.value p.value
## ar1         0.9289 0.0173  53.6113       0
## intercept  61.3932 6.1934   9.9128       0
## SL5       -14.6892 1.3858 -10.6001       0
##
## sigma^2 estimated as 91.74432 on 445 degrees of freedom
##
## AIC = 7.379176  AICc = 7.379296  BIC = 7.415825
##
```

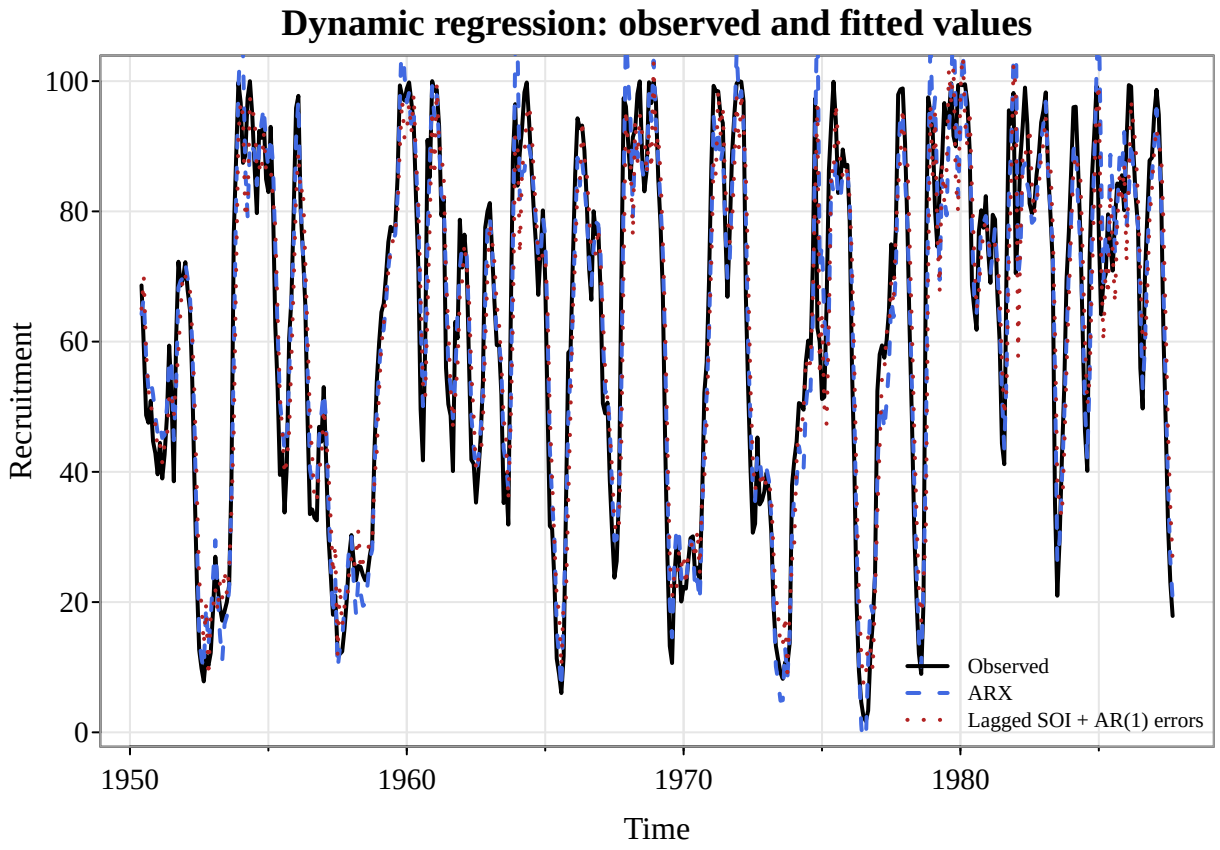



```r
# Obtain fitted values on the same observation period.
fitted.arx <- ts(fitted(fit.arx), start = start(fish), frequency = frequency(fish))

# Observed value minus fitted innovation gives the fitted prediction.
fitted.regAR <- fish[, "REC"] - residuals(fit.regAR$fit)

# Compare observed recruitment and in-sample fitted values.
tsplot(fish[, "REC"], col = "black", lwd = 2,
  ylab = "Recruitment", main = "Dynamic regression: observed and fitted values")
lines(fitted.arx, col = "royalblue", lwd = 2, lty = 2)
lines(fitted.regAR, col = "firebrick", lwd = 2, lty = 3)

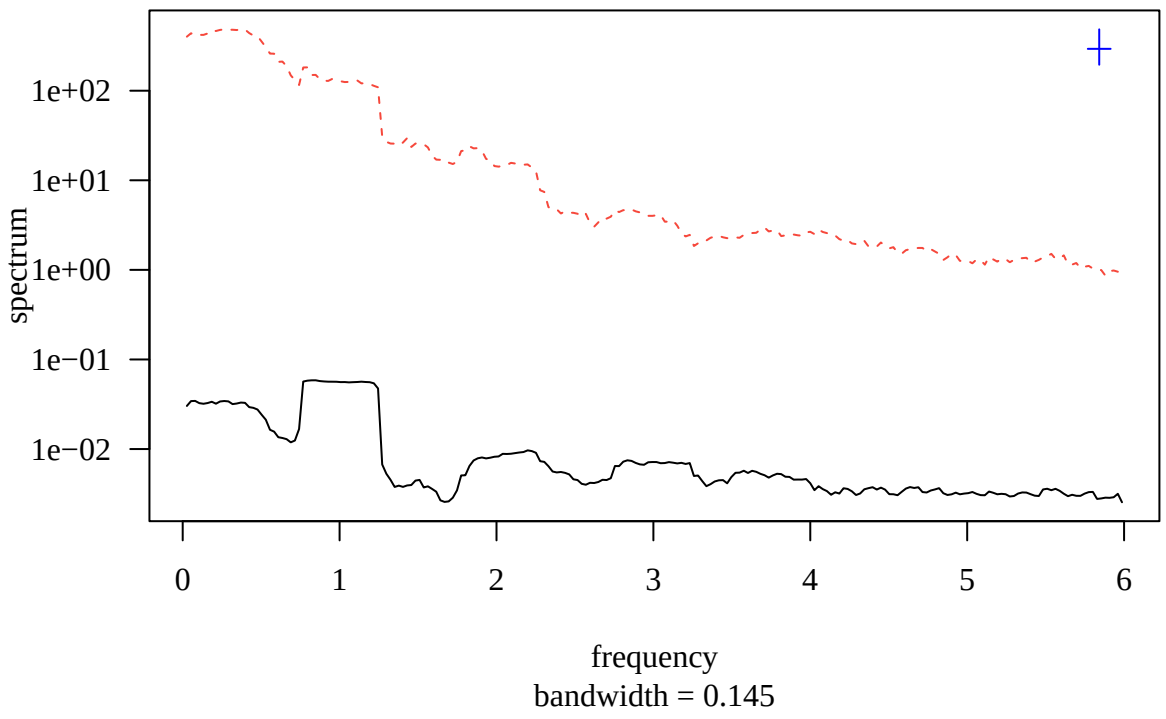
legend("bottomright",
  legend = c("Observed", "ARX", "Lagged SOI + AR(1) errors"),
  col = c("black", "royalblue", "firebrick"),
  lty = c(1, 2, 3), lwd = 2, bty = "n", cex = 0.7)
```




```
# Extend these examples:
# 1. Explore plausible SOI delays, such as 4-8 months.
# 2. Compare single-lag and short distributed-lag responses.
# 3. Consider alternative ARMA errors for Model B.
# 4. Inspect residual diagnostics before accepting a model.
# 5. Evaluate forecasts on held-out data against a recruitment-only baseline
# to assess whether SOI improves prediction.
```

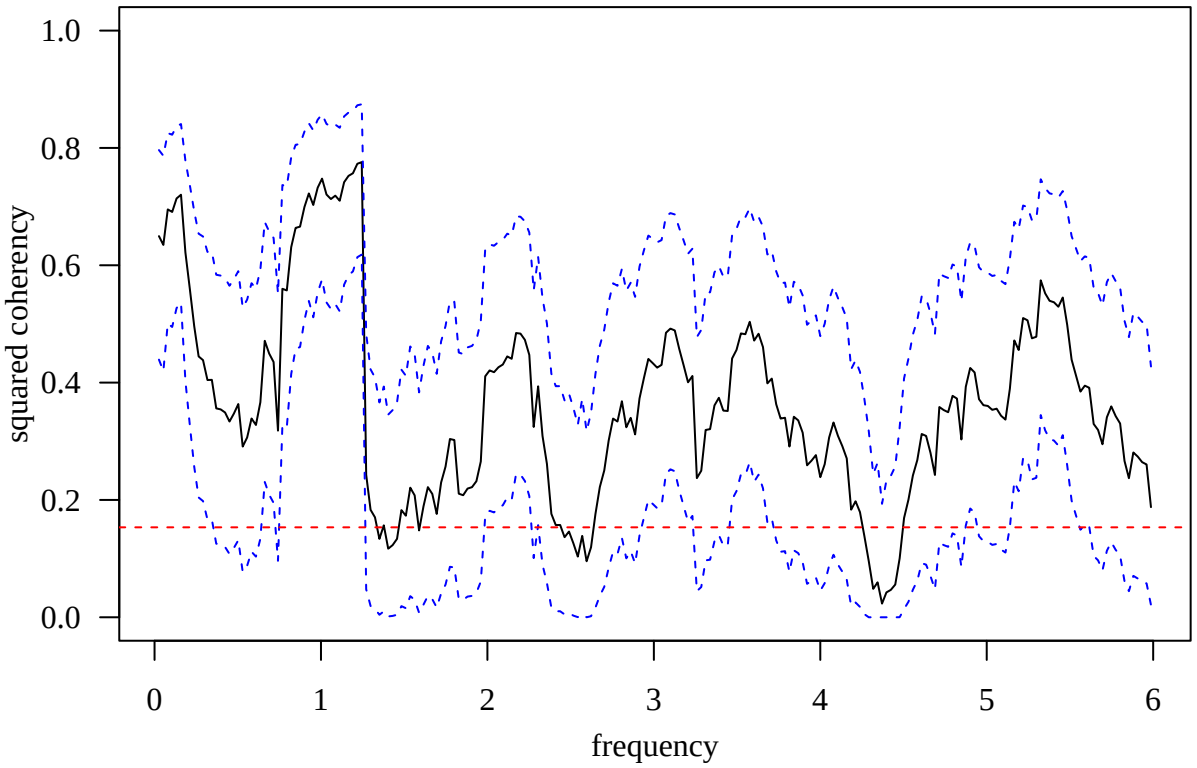
Estimating the Cross-Spectrum and Coherence

```
par(las = 1, family = "serif")
s = spectrum(cbind(soi, rec), kernel("daniell", 9), taper = 0, fast = FALSE)
```




```r
par(las = 1, mar = c(4, 4, 2, 0.6), mgp = c(2.2, 1, 0), family = "serif")
plot(s, plot.type = "coh", ci.lty = 2, main = "")

# Add a horizontal line for the 95% significance level
# ----------------------------------------------------
# Compute the F-quantile for the coherence significance test.
f = qf(.95, 2, s$df - 2);
# The corresponding coherence threshold:
abline(h = f / ((s$df - 2) / 2 + f), col = "red", lty = 2)
```



We estimate the squared coherence between SOI and recruitment by smoothing their auto-spectra and cross-spectrum with a Daniell kernel. The coherence plot shows which frequencies have stronger linear association between the two series. The red horizontal line is a pointwise significance threshold for testing zero coherence. The spectrum() example uses a 95% threshold, and the mvspec() example uses a stricter 99.9% threshold. Values above the line provide evidence of an association at that frequency.

Lagged Regression in the Frequency Domain

The time-domain approach uses prewhitening and cross-correlation to identify candidate response lags. The frequency-domain approach describes the same linear relationship through a **transfer function**, which characterizes how the input is amplified, attenuated, and shifted in phase at different frequencies.

For jointly stationary input and output series, consider

$$Y_t - \mu_Y = \sum_{j=-\infty}^{\infty} \beta_j (X_{t-j} - \mu_X) + V_t,$$

where V_t is uncorrelated with the input at all lags. Here, β_j describes the impulse-response coefficient at lag j:

- $j > 0$: earlier input values contribute to the current output.
- $j = 0$: the input contributes contemporaneously.
- $j < 0$: future input values enter the two-sided representation.

The transfer function is

$$H(\omega) = \sum_{j=-\infty}^{\infty} \beta_j e^{-2\pi i \omega j}.$$

Using the cross-spectrum,

$$f_{YX}(\omega) = H(\omega) f_X(\omega),$$

we obtain

$$H(\omega) = \frac{f_{YX}(\omega)}{f_X(\omega)}, \qquad f_X(\omega) > 0.$$

Finally, the inverse Fourier transform recovers the response coefficients:

$$\beta_j = \int_{-1/2}^{1/2} H(\omega) e^{2\pi i \omega j} \, d\omega.$$

Thus, the basic workflow is:

1. Estimate and smooth the auto- and cross-spectra.
2. Estimate the transfer function using the spectral ratio.
3. Invert the transform to estimate the impulse response.
4. Examine the response pattern and construct a parsimonious approximation.

Unlike the raw CCF, this approach accounts for input persistence through the input spectrum in the denominator. Smoothing is important because unsmoothed spectral ratios are unstable, especially where the input has little spectral power.

Connection to coherence. Squared coherence,

$$C_{XY}(\omega) = \frac{|f_{YX}(\omega)|^2}{f_X(\omega) f_Y(\omega)},$$

describes the strength of the linear relationship at each frequency. The transfer function adds information about its gain and phase. Neither quantity alone establishes causality.

Example: SOI and Recruitment We use `astsa::LagReg()` to illustrate the frequency-domain approach. `SOI` is the input and `recruitment` is the output.

The tuning arguments have different roles:

Argument	Role
`L = 15`	Controls spectral smoothing through the smoothing span.
`M = 32`	Controls the finite lag approximation; the implementation evaluates coefficients at lags $-15, \ldots, 15$.
`threshold = 6`	Retains coefficients whose absolute magnitudes are at least 6.

The threshold is a **coefficient-size cutoff**, not a significance level. Its interpretation depends on the units of the input and output.

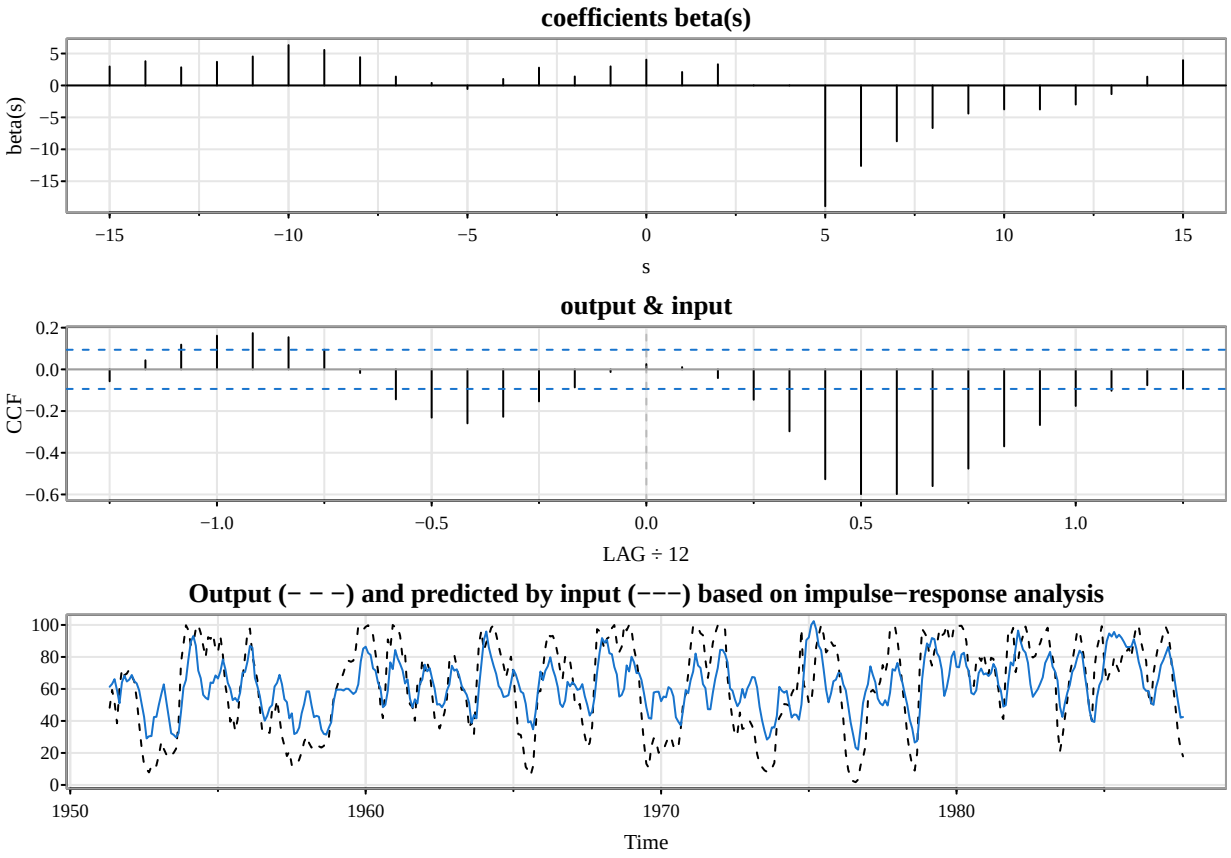
We first estimate the unthresholded response, then retain sufficiently large coefficients at nonnegative lags. We construct fitted values explicitly so that coefficient β_j multiplies X_{t-j}.

```
library(astsa)
par(las = 1, family = "serif")

# Step 1: Estimate the impulse response without thresholding.
lagreg.full <- astsa::LagReg(input = soi, output = rec,
                             L = 15,           # spectral smoothing span
                             M = 32,           # even-valued lag approximation size
                             threshold = 0, inverse = FALSE)
```

```
## INPUT: soi OUTPUT: rec   L = 15    M = 32
##
## The coefficients beta(0), beta(1), beta(2) ... beta(M/2-1) are
##
## 4.03743 2.103372 3.31812 0.01247538 0.005194443 -18.90914 -12.60978 -8.746491
## -6.670373 -4.404543 -3.748336 -3.760936 -2.991477 -1.355261 1.375379 3.955252
##
##
##
## The coefficients beta(0), beta(-1), beta(-2) ... beta(-M/2+1) are
##
## 4.03743 2.987159 1.409949 2.788212 1.017324 -0.5528797 0.402843 1.389537
## 4.426287 5.563582 6.315986 4.540402 3.703423 2.840445 3.798354 2.974338

## The positive lags, at which the coefficients are large
## in absolute value, and the coefficients themselves, are:
##       lag s       beta(s)
##  [1,]     0   4.037429653
##  [2,]     1   2.103372494
##  [3,]     2   3.318120131
##  [4,]     3   0.012475378
##  [5,]     4   0.005194443
##  [6,]     5 -18.909139762
##  [7,]     6 -12.609781033
##  [8,]     7  -8.746490655
##  [9,]     8  -6.670373374
## [10,]     9  -4.404543150
## [11,]    10  -3.748336347
## [12,]    11  -3.760935651
## [13,]    12  -2.991477145
## [14,]    13  -1.355261381
## [15,]    14   1.375378996
## [16,]    15   3.955252475
```




```
##
## The prediction equation is
## rec(t) = alpha + sum_s[ beta(s)*soi(t-s) ], where alpha = 66.13573
## MSE = 401.7428
```

```r
# Step 2: Extract the response coefficients.
# In this implementation, betas contains lag and coefficient.
beta <- lagreg.full$betas
colnames(beta) <- c("lag", "coefficient")

# Step 3: Retain current/past-input terms above the cutoff.
cutoff <- 6
keep <- beta[, "lag"] >= 0 & abs(beta[, "coefficient"]) >= cutoff

selected <- beta[keep, , drop = FALSE]

if (nrow(selected) == 0L) {
  stop("No coefficients retained; try a smaller cutoff.")
}

knitr::kable(selected, digits = 3, row.names = FALSE,
  col.names = c("Response lag (months)", "Coefficient"),
  caption = "Retained impulse-response coefficients"
)
```

Table 2: Retained impulse-response coefficients

Response lag (months)	Coefficient
5	-18.909
6	-12.610
7	-8.746
8	-6.670

```
# Step 4: Align the input and output.
aligned <- ts.intersect(REC = rec, SOI = soi)
y <- aligned[, "REC"]
x <- aligned[, "SOI"]

lags <- as.integer(selected[, "lag"])
b <- selected[, "coefficient"]

# Explicitly align lag j with X_{t-j}.
dat <- ts.intersect(y, xlag = stats::lag(x, -lags[1]))

if (length(lags) > 1L) {
  for (j in 2:length(lags)) {dat <- ts.intersect(dat, stats::lag(x, -lags[j]))
  }
}

# Restore the means after applying the centered response.
alpha <- mean(y) - mean(x) * sum(b)

fitted.values <- ts(alpha + drop(dat[, -1, drop = FALSE] %*% b), start = start(dat), frequency = freque

errors <- dat[, 1] - fitted.values

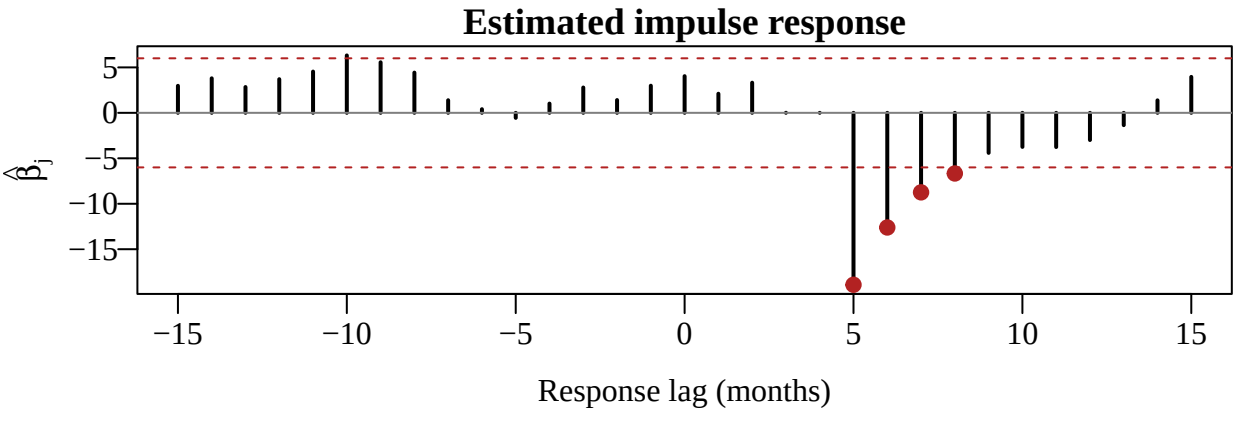
cat("Estimated intercept:", round(alpha, 3), "\n")
```

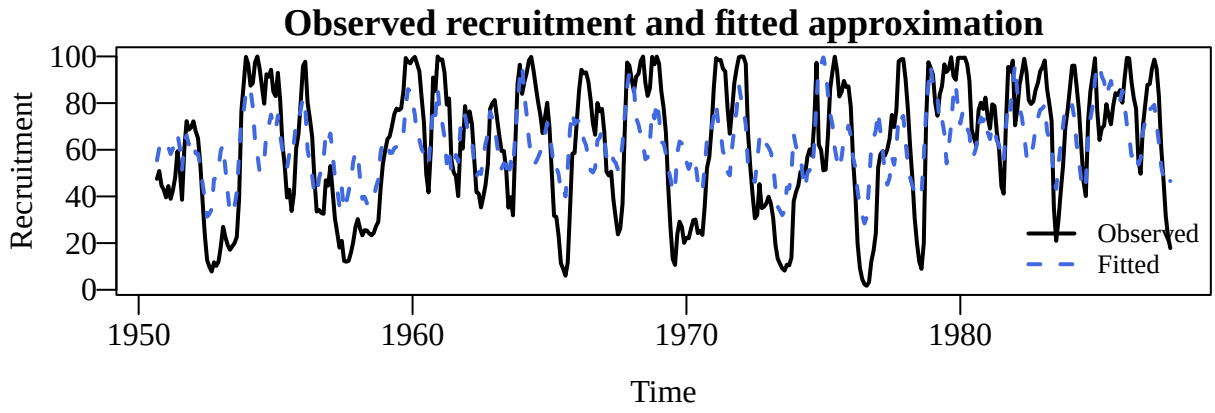
```
## Estimated intercept: 66.019
```

```
# Step 5: Display the response and fitted approximation.
old.par <- par(mfrow = c(2, 1), mar = c(3.5, 3.5, 1.3, 0.5), mgp = c(2, 0.5, 0), family = "serif")

plot(beta[, "lag"], beta[, "coefficient"], type = "h", lwd = 2,
  xlab = "Response lag (months)", ylab = expression(hat(beta)[j]), main = "Estimated impulse response")
abline(h = 0, col = "gray50")
abline(h = c(-cutoff, cutoff), col = "firebrick", lty = 2)
points(lags, b, col = "firebrick", pch = 19)

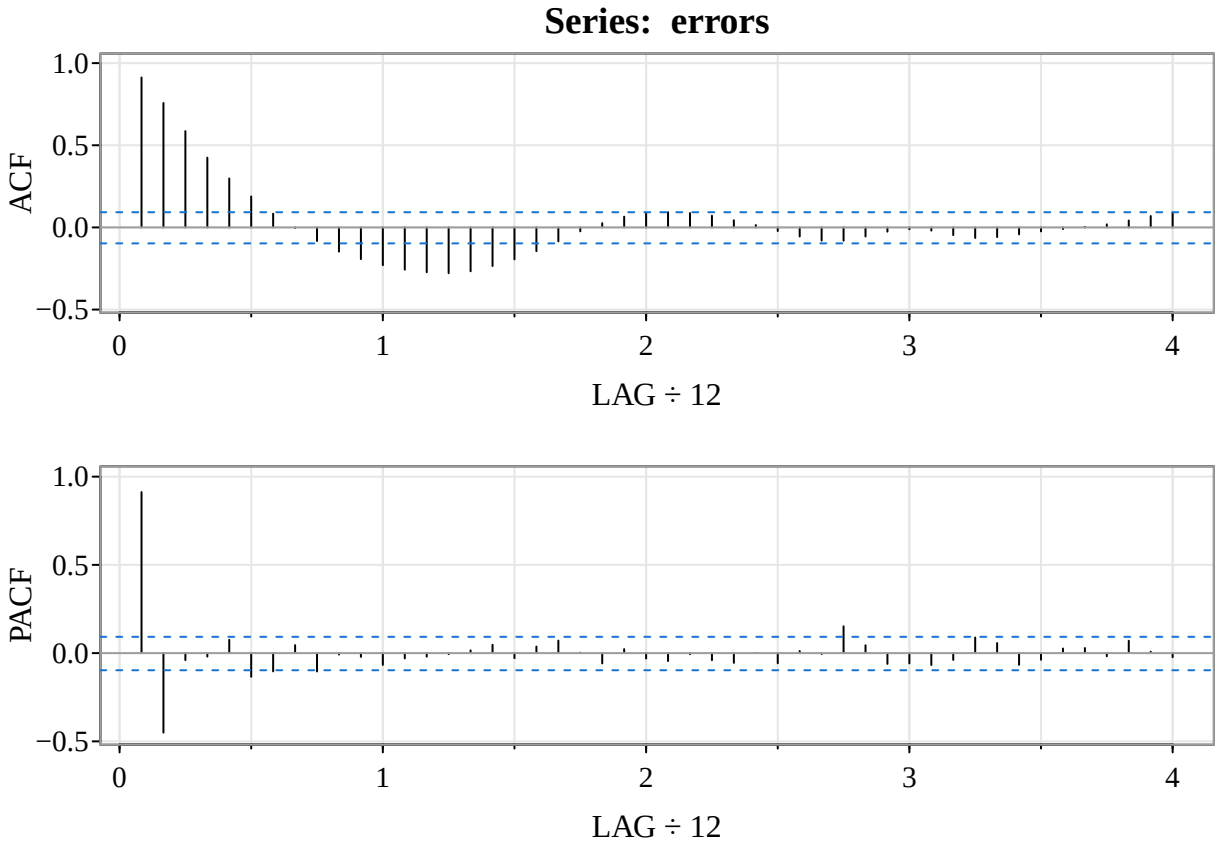
ts.plot(dat[, 1], fitted.values, col = c("black", "royalblue"), lty = c(1, 2), lwd = 2,
  ylab = "Recruitment", main = "Observed recruitment and fitted approximation")
legend("bottomright", legend = c("Observed", "Fitted"),
  col = c("black", "royalblue"),
  lty = c(1, 2), lwd = 2, bty = "n", cex = 0.8)
```






```
par(old.par)

# Step 6: Examine remaining serial dependence.
invisible(acf2(errors))
```



Interpreting the Estimated Response If the retained coefficients match the reported example, the resulting approximation is

$$Y_t = 66.02 - 18.91X_{t-5} - 12.61X_{t-6} - 8.75X_{t-7} - 6.67X_{t-8} + V_t.$$

This suggests a **negative response concentrated approximately five to eight months after an SOI input**, with the largest estimated magnitude at five months and smaller magnitudes afterward.

Under the fitted filter interpretation, an isolated one-unit SOI impulse is associated with a recruitment response of approximately -18.91 five months later, followed by smaller negative responses at months six through eight. These coefficients describe a distributed response; they are not separate estimates from four unrelated regressions.

Use the coefficients printed by your run when reporting the final equation. They can change with preprocessing, smoothing, approximation size, and threshold.

The remaining process V_t represents unexplained variation. It should not be assumed to be white noise: inspect its ACF and consider additional noise modeling if dependence remains.

Two-Sided Filtering versus Causal Prediction The spectral ratio f_{YX}/f_X solves an **unrestricted two-sided linear projection** problem. Although we retain only current and past input terms in this example, discarding negative-lag coefficients does not generally produce the optimal constrained causal predictor.

Also, a filter that uses only past inputs is "causal" in the time-ordering sense. This does not establish that SOI has a causal scientific effect on recruitment.

The displayed fitted values are an in-sample approximation based on the estimated input response. They are not evidence of out-of-sample forecast accuracy.

Suggested Extensions This example provides a starting point for further investigation:

1. **Vary the smoothing span.** Compare several values of `L`. More smoothing generally stabilizes estimates but can blur narrow spectral features and change the recovered response.
2. **Vary the response window.** Change `M` to investigate shorter or longer lag approximations. Check whether the main response remains concentrated near the same months.
3. **Assess threshold sensitivity.** Compare the unthresholded response with several coefficient cutoffs. Do not interpret retained coefficients as automatically statistically significant.
4. **Check preprocessing sensitivity.** Repeat the analysis with detrended SOI. Use consistent preprocessing when comparing time-domain and frequency-domain results.
5. **Evaluate predictive value.** Reserve later observations before selecting tuning choices. Compare against a recruitment-only model, using only information available at each forecast origin.

Focus on whether the **response timing, sign, and predictive value** remain stable across reasonable choices, rather than selecting the configuration that gives the closest in-sample fit.